Neighbourhood methods: nearest neighbours, Parzen windows and distances

Prof. Dr. Emre Onur Kahya · Department of Physics Engineering, Istanbul Technical University

Neighbourhood methods predict at a point by letting nearby training examples vote. They need no training, make no assumption about the shape of the target function, and handle classification and regression with one weighted-average formula, whose denominator — the total weight near the query — is itself a density estimate. This note develops k-nearest neighbours and Parzen windows (kernel smoothing and kernel density estimation), discusses the distances they rely on, and shows that the same ideas power retrieval in today's AI systems and the attention mechanism of transformers. A small dataset of eight points is followed through every method by hand; each algorithm is given as pseudocode with its running time. Model selection follows Note 4; the geometry of high dimension follows Note 3.

1 Let the neighbours vote

Fix a distance $d(\cdot,\cdot)$ on the input space, Euclidean unless stated otherwise. For a query $\mathbf{x}$, let $N_k(\mathbf{x})$ be the set of indices of the k training points closest to $\mathbf{x}$. The *k-nearest-neighbour* (k-NN) prediction averages their targets. All three tasks fit one formula,

$$\bar{\mathbf{y}}(\mathbf{x}) = \frac{\sum_{i=1}^{n} w_i(\mathbf{x})\,\mathbf{t}_i}{\sum_{i=1}^{n} w_i(\mathbf{x})}, \qquad w_i(\mathbf{x}) = \mathbf{1}\{i \in N_k(\mathbf{x})\}, \tag{1}$$

with a task-dependent target encoding $\mathbf{t}_i$ and read-out:

- **regression**: $\mathbf{t}_i = y_i$ and $f(\mathbf{x}) = \bar{\mathbf{y}}(\mathbf{x})$, the mean target of the neighbours (with several real outputs, $\mathbf{t}_i$ is the vector of targets and the average is taken component by component);
- **classification with m classes**: $\mathbf{t}_i = \text{onehot}(y_i) \in \{0,1\}^m$, so $\bar{\mathbf{y}}(\mathbf{x})$ is the vector of class frequencies among the neighbours — an estimate of $P(Y = \cdot \mid \mathbf{X} = \mathbf{x})$ — and $f(\mathbf{x}) = \arg\max_c \bar{y}_c(\mathbf{x})$, the majority vote;
- **binary classification** with $y_i \in \{-1,+1\}$: $f(\mathbf{x}) = \operatorname{sign} \bar{y}(\mathbf{x})$.

Like the histograms of Note 3, k-NN replaces $\mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$ by an average over a neighbourhood, but the neighbourhood is now centred on the query and adapts its size to the local density of data: small where examples are plentiful, large where they are sparse. No cell is ever empty.

Neighbourhood methods are called *non-parametric*: they assume nothing about the form of f, and their "parameters" are the training set itself, which must be stored. Their hyperparameters — k and the distance — are chosen on validation data as in Note 4. They are also called *lazy* (or memory-based) learners: all the work is postponed until a query arrives.

1.1 One formula, three read-outs

Equation (1) looks more general than "average the neighbours", so let us check that it is the same thing. Exactly k of the weights equal one and the others are zero, so $\sum_i w_i(\mathbf{x}) = k$ and

$$\bar{\mathbf{y}}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} \mathbf{t}_i.$$

- *Regression.* The mean is not an arbitrary choice: it is the constant c that fits the neighbours best in squared error. Setting the derivative of $J(c) = \sum_{i \in N_k(\mathbf{x})} (y_i - c)^2$ to zero,

$$J'(c) = -2 \sum_{i \in N_k(\mathbf{x})} (y_i - c) = 0 \quad \Longrightarrow \quad c = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i.$$

  This is the local, finite-sample version of the statement of Note 1, Section 4, that $\mathbb{E}[Y \mid \mathbf{X} = \mathbf{x}]$ minimises the expected squared error.

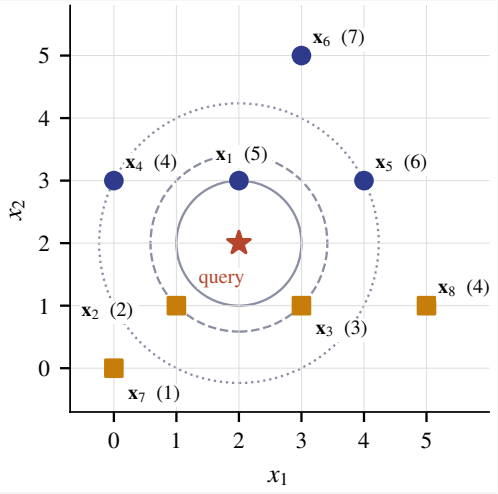
- *Classification.* Component c of the average of one-hot vectors is $\bar{y}_c = k_c/k$, where k_c is the number of neighbours of class c. These numbers are non-negative and sum to one, so they form a probability vector, our estimate $\hat{P}(Y = c \mid \mathbf{X} = \mathbf{x}) = k_c/k$. The arg max picks the most frequent class, which is also the constant prediction that misclassifies the fewest neighbours.

- *Binary classification with ±1 labels.* With k_+ positive and $k_- = k - k_+$ negative neighbours, $\bar{y} = (k_+ - k_-)/k = (2k_+ - k)/k$, which is positive exactly when $k_+ > k/2$: the sign rule is the majority vote again, and $(1 + \bar{y})/2 = k_+/k$ is the estimated probability of the positive class.

1.2 A worked example

Figure 1, inside the box below, shows the small dataset used throughout this note: eight points in the plane, each with a class (A or B) and, for regression, a real target y_i written in parentheses. The query is $\mathbf{x} = (2, 2)$. Squared distances suffice for ranking, since the square root is increasing. The table is sorted by distance, not by index (so $\mathbf{x}_7$ comes before $\mathbf{x}_6$).

Worked example — k-NN by hand for $k = 1, 3, 5$



i	$\mathbf{x}_i$	class	y_i	$\lVert\mathbf{x}_i - \mathbf{x}\rVert^2$	rank
1	$(2, 3)$	A	5	1	1
2	$(1, 1)$	B	2	2	2–3
3	$(3, 1)$	B	3	2	2–3
4	$(0, 3)$	A	4	5	4–5
5	$(4, 3)$	A	6	5	4–5
7	$(0, 0)$	B	1	8	6
6	$(3, 5)$	A	7	10	7–8
8	$(5, 1)$	B	4	10	7–8

Figure 1. The running example: class A (circles), class B (squares), targets y_i in parentheses, query $\mathbf{x} = (2, 2)$ (star). The grey rings pass through the first, third and fifth nearest neighbours.

For instance $\|\mathbf{x}_6 - \mathbf{x}\|^2 = (3-2)^2 + (5-2)^2 = 10$. Reading the sorted list from the top:

- $k = 1$: the nearest point is $\mathbf{x}_1$ (class A). Prediction A, with $\hat{P}(\mathrm{A} \mid \mathbf{x}) = 1$; regression $f(\mathbf{x}) = 5$.
- $k = 3$: neighbours $\{1, 2, 3\}$ with classes A, B, B. Votes A : B $= 1 : 2$, prediction B, $\hat{P}(\mathrm{A} \mid \mathbf{x}) = 1/3$; regression $f(\mathbf{x}) = (5 + 2 + 3)/3 = 10/3 \approx 3.33$.
- $k = 5$: neighbours $\{1, \ldots, 5\}$. Votes $3 : 2$, prediction A, $\hat{P}(\mathrm{A} \mid \mathbf{x}) = 3/5$; regression $f(\mathbf{x}) = (5 + 2 + 3 + 4 + 6)/5 = 4$.

The query sits between the classes, and the answer flips twice as k grows. Nothing is wrong: k is a hyperparameter, and only validation data can tell which value generalises (Section 2.1).

1.3 Ties

Two kinds of ties occur, and an implementation must decide both.

- *Distance ties*: several points are at the same distance as the k-th neighbour. In the example, $\mathbf{x}_4$ and $\mathbf{x}_5$ are both at squared distance 5, so "the four nearest neighbours" is not well defined. Common conventions are to keep the point with the smaller index (what a stable sort does) or to include all tied points, letting the neighbourhood grow slightly.
- *Vote ties*: two classes receive the same number of votes. With $k = 4$ in the example, whichever of $\mathbf{x}_4$, $\mathbf{x}_5$ is kept, the votes are A : B $= 2 : 2$.

For two classes an odd k prevents vote ties; with $m \geq 3$ classes no $k > 1$ does (two classes can tie at $k = 2$, three can share $k = 3$ votes equally). Three common rules for vote ties, applied to $k = 4$:

1. *drop the farthest neighbour* and vote again with $k - 1$, repeating if needed: here $k = 3$ gives B;
2. *prefer the tied class with the nearest member*: $\mathbf{x}_1$ (A) is nearer than any B point, so A;
3. *weight the votes by inverse distance*, $w_i = 1/\|\mathbf{x}_i - \mathbf{x}\|$ for $i \in N_k(\mathbf{x})$: A collects $1/1 + 1/\sqrt{5} = 1.447$ and B collects $2/\sqrt{2} = 1.414$, so A.

Different rules give different answers on a genuinely ambiguous query; what matters is to fix one and use it consistently. Rule 1 is the one used in Algorithms 1 and 2 below. Rule 3, distance-weighted k-NN (a one-line change of Algorithm 1), is a first step towards the soft weights of Section 3.

Decision regions. For $k = 1$, each training point owns the region of space closer to it than to any other point, its *Voronoi cell*. The classifier predicts the label of the cell's owner, so the decision boundary is made of pieces of perpendicular bisectors between differently labelled neighbours (Figure 2). It can be arbitrarily complex: one mislabelled point carves out its own island.

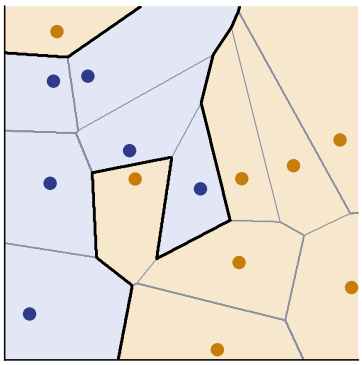


Figure 2. The 1-NN classifier for 14 points. Thin lines: Voronoi cells; thick line: decision boundary; shading: predicted class.

The choice of k. Figure 3 shows k-NN on 300 points from two interleaved half-moons. With $k = 1$ the training error is zero and the boundary follows every noisy point; the test error is 9.2%. With $k = 15$ the boundary is smooth and training and test errors agree at about 6%. With $k = 151$, half the training set votes everywhere, the boundary is too simple to follow the moons and both errors rise to about 19%. Small k means low bias and high variance; large k the reverse (Note 4, Section 6). In the limit $k = n$ every query receives the overall majority class (and, in regression, the overall mean target).

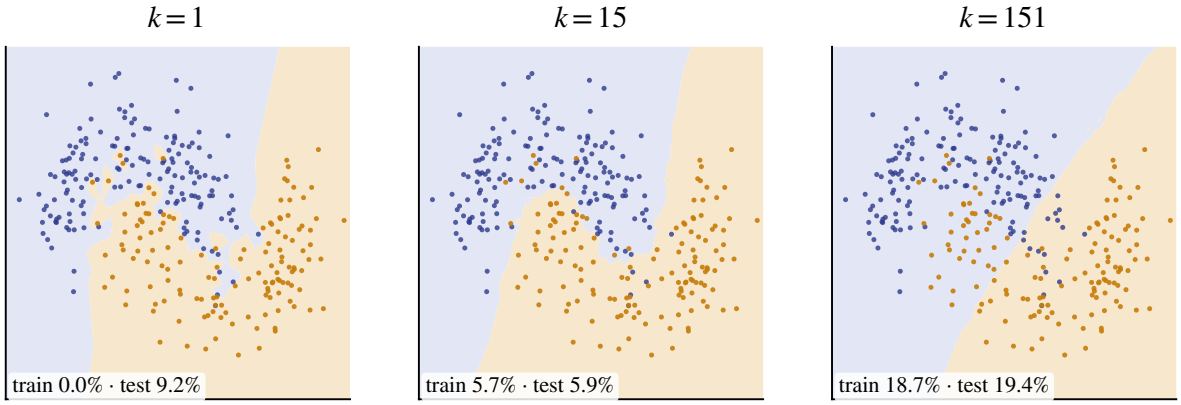


Figure 3. k-NN classification of 300 training points; errors on the training set and on 3000 test points.

What theory guarantees. With enough data, nearest neighbours are close to the query and the method approaches the ideal. As $n \to \infty$, the error of 1-NN is at most twice the Bayes error (for two classes, $R^* \le R_{1\text{-NN}} \le 2R^*(1 - R^*)$) [4], and k-NN is *consistent* — its risk converges to the Bayes risk for every distribution — if $k \to \infty$ while $k/n \to 0$ [19]. The two conditions have a plain meaning: $k \to \infty$ makes the vote an average over many points (variance $\to 0$), and $k/n \to 0$ makes the neighbourhood shrink around $\mathbf{x}$ (bias $\to 0$). The catch is the rate: in d dimensions the distance to the nearest neighbour shrinks only like $n^{-1/d}$ — a ball of radius r around $\mathbf{x}$ holds about nr^d of the points (up to a constant), and setting this equal to 1 gives $r \sim n^{-1/d}$ — and Note 3, Section 3, showed what that means for $d = 100$.

2 Computing neighbours

The algorithm is short (Algorithm 1). Training stores $X \in \mathbb{R}^{n \times d}$ and the targets. Prediction computes the distances from the query to all training points, selects the k smallest and reads out the vote or the mean.

The leader $\hat{c}$ changes only when a class strictly overtakes it, so among classes tied at the end it keeps the one that reached the winning count first. This is exactly tie rule 1 of Section 1.3. Indeed, the leader always holds a maximal count, and it changes only at a step where another class becomes the *unique* maximum. Let $r \le k$ be the last step at which a single class holds the maximal count (step 1 always qualifies). At step r that class is the leader, and since no class becomes the unique maximum afterwards, the leader never changes again. Tie rule 1 drops the farthest neighbours until the tie disappears, which takes it back to the same step r and the same class. In the weighted variant (rule 3) the counts v_c become sums of inverse distances; a neighbour at distance zero is handled first by returning its class directly. In numpy, a batch of queries is handled with the distance matrix of Note 1, Section 6:

Algorithm 1 k-nearest neighbours, brute force: classification and regression

Require: training inputs $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, targets $y_1, \dots, y_n$ (classes $1..m$ or real numbers), number of neighbours $k \le n$

function TRAIN(D_{train})
  store X and $\mathbf{y}$ ▷ nothing is fitted
end function
function NEIGHBOURS($\mathbf{x}, k$)
  for $i = 1, \dots, n$ **do**
    $\delta_i \leftarrow \|\mathbf{x}_i - \mathbf{x}\|^2$ ▷ squared distance: same ranking, no square root
  end for
  $N \leftarrow$ indices of the k smallest δ_i, sorted by δ_i ▷ partial selection
  return N
end function
function CLASSIFY($\mathbf{x}$)
  $(i_1, \dots, i_k) \leftarrow$ NEIGHBOURS($\mathbf{x}, k$)
  $v_c \leftarrow 0$ for $c = 1, \dots, m$; $\hat{c} \leftarrow$ none ▷ $\hat{c}$: current leader
  for $r = 1, \dots, k$ **do**
    $v_{y_{i_r}} \leftarrow v_{y_{i_r}} + 1$ ▷ weighted variant: add $1/\left\|\mathbf{x}_{i_r} - \mathbf{x}\right\|$ (rule 3)
    if $\hat{c}$ is none **or** $v_{y_{i_r}} > v_{\hat{c}}$ **then** $\hat{c} \leftarrow y_{i_r}$ ▷ strict ">": tie rule 1
  end for
  return $\hat{c}$ ▷ v_c/k estimates $P(Y = c \mid \mathbf{X} = \mathbf{x})$
end function
function REGRESS($\mathbf{x}$)
  $N \leftarrow$ NEIGHBOURS($\mathbf{x}, k$)
  return $\frac{1}{k} \sum_{i \in N} y_i$
end function

```
def knn_classify(Xtr, ytr, Xq, k, m):        # ytr: integer labels 0..m-1
    D = sq_distances(Xq, Xtr)                 # (n_q, n), Note 1
    idx = np.argpartition(D, k - 1, axis=1)[:, :k]   # k nearest, unordered
    votes = np.eye(m)[ytr[idx]].sum(axis=1)   # (n_q, m) class counts
    return votes.argmax(axis=1)               # ties: smallest class index
```

Partial selection (`argpartition`, "quickselect") finds the k smallest of n numbers in $O(n)$ average time; a full sort would cost $O(n \log n)$, and a heap of size k costs $O(n \log k)$. Note that this short version breaks vote ties by class index, a different (and cruder) convention than Algorithm 1.

Running time — k-NN, brute force (Algorithm 1)

Notation: n training examples, d features, m classes, k neighbours, n_q queries.

- *Training*: $O(1)$, or $O(nd)$ to copy the data. Nothing is computed.
- *Prediction per query*: $O(nd)$ for the n distances (each a sum of d squares), $O(n)$ for the selection (plus $O(k \log k)$ to sort the k winners), $O(k+m)$ for the vote or the mean. Total $O(nd + k \log k + m)$, which is $O(nd + m)$ in the usual case $k \log k \le nd$: the distances dominate.
- *Memory*: $O(nd)$ for the stored training set, $O(n)$ working space per query; a batch of n_q queries needs the $n_q \times n$ distance matrix, so large batches are processed in chunks.
- *Distance-weighted variant* (rule 3): k extra square roots and divisions, so the cost is unchanged.
- *In practice*: the $O(n_q nd)$ distance computation is one matrix product (Note 1, Section 6), which BLAS libraries and GPUs execute at close to peak speed. Still, with $n = 10^6$ and $d = 100$, one query costs 10^8 multiply–adds and the stored data take 400 MB in 32-bit floats: training is free,

prediction is expensive — the opposite of most models.

2.1 Choosing k by leave-one-out

Any of the validation schemes of Note 4, Section 4, can select k. For k-NN, leave-one-out cross-validation is unusually cheap. Leaving $\mathbf{x}_i$ out and predicting it from the rest only requires the neighbours of $\mathbf{x}_i$ among the other points, i.e. its own row of the distance matrix D with the diagonal entry D_{ii} removed. Moreover, once the $k_{\max}$ nearest neighbours of $\mathbf{x}_i$ are sorted, the predictions for *all* $k = 1, \dots, k_{\max}$ follow by walking down the list once and updating the running vote (Algorithm 2). One pass gives the whole validation curve.

Algorithm 2 Leave-one-out choice of k for k-NN classification

Require: training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^{n}$ with classes $1..m$; largest candidate $k_{\max} \le n-1$
Ensure: the $k \in \{1, \dots, k_{\max}\}$ with the smallest leave-one-out error, and all the error estimates

$e_k \leftarrow 0$ for $k = 1, \dots, k_{\max}$
for $i = 1, \dots, n$ **do**
  $\delta_j \leftarrow \|\mathbf{x}_j - \mathbf{x}_i\|^2$ for all j; $\delta_i \leftarrow \infty$ ▷ leave $\mathbf{x}_i$ out
  $(j_1, \dots, j_{k_{\max}}) \leftarrow$ indices of the $k_{\max}$ smallest δ_j, sorted
  $v_c \leftarrow 0$ for all c; $\hat{c} \leftarrow$ none
  for $r = 1, \dots, k_{\max}$ **do** ▷ prediction with $k = r$ neighbours
    $v_{y_{j_r}} \leftarrow v_{y_{j_r}} + 1$
    if $\hat{c}$ is none **or** $v_{y_{j_r}} > v_{\hat{c}}$ **then** $\hat{c} \leftarrow y_{j_r}$
    $e_r \leftarrow e_r + \mathbf{1}\{\hat{c} \ne y_i\}$
  end for
end for
return $k^* = \arg\min_k e_k$ and the error estimates e_k/n

Running time — leave-one-out choice of k (Algorithm 2)

For each of the n points: $O(nd)$ distances, $O(n + k_{\max} \log k_{\max})$ selection and sorting, $O(k_{\max})$ for the running vote. Total $O(n^2 d + n k_{\max} \log k_{\max})$ time — the cost of predicting the n training points once — for the whole curve $k = 1, \dots, k_{\max}$. Memory $O(nd + n + k_{\max})$ if rows are computed one at a time. Without the running vote, trying each k separately would multiply the cost by $k_{\max}$. For n beyond about 10^5 the n^2 factor bites, and a single validation split (cost $O(n_{\text{val}}\, nd)$) or approximate neighbours are used instead.

Worked example — leave-one-out choice of k on the eight points

Run Algorithm 2 on the data of Figure 1 (classes only) with $k_{\max} = 3$. For each left-out point the table lists its three nearest other points (distances computed from the coordinates in the table of Figure 1; distance ties broken by the smaller index, as a stable sort does) and the running leader $\hat{c}$ after $r = 1, 2, 3$ neighbours.

left out	class	neighbours (class)	$k=1$	$k=2$	$k=3$	errors
$\mathbf{x}_1$	A	$\mathbf{x}_4$ (A), $\mathbf{x}_5$ (A), $\mathbf{x}_2$ (B)	A	A	A	—
$\mathbf{x}_2$	B	$\mathbf{x}_7$ (B), $\mathbf{x}_3$ (B), $\mathbf{x}_1$ (A)	B	B	B	—
$\mathbf{x}_3$	B	$\mathbf{x}_2$ (B), $\mathbf{x}_8$ (B), $\mathbf{x}_1$ (A)	B	B	B	—
$\mathbf{x}_4$	A	$\mathbf{x}_1$ (A), $\mathbf{x}_2$ (B), $\mathbf{x}_7$ (B)	A	A	B	$k=3$
$\mathbf{x}_5$	A	$\mathbf{x}_1$ (A), $\mathbf{x}_3$ (B), $\mathbf{x}_6$ (A)	A	A	A	—
$\mathbf{x}_6$	A	$\mathbf{x}_1$ (A), $\mathbf{x}_5$ (A), $\mathbf{x}_4$ (A)	A	A	A	—
$\mathbf{x}_7$	B	$\mathbf{x}_2$ (B), $\mathbf{x}_4$ (A), $\mathbf{x}_3$ (B)	B	B	B	—
$\mathbf{x}_8$	B	$\mathbf{x}_3$ (B), $\mathbf{x}_5$ (A), $\mathbf{x}_1$ (A)	B	B	A	$k=3$

Take $\mathbf{x}_4$: after $r = 1$, $v_A = 1$ and the leader is A; after $r = 2$, $v_B = 1$ ties but does not exceed v_A, so A stays; after $r = 3$, $v_B = 2 > 1$ and the leader becomes B, a mistake. The error counts are $e_1 = 0$, $e_2 = 0$, $e_3 = 2$, i.e. leave-one-out errors 0, 0 and $2/8 = 0.25$, and the arg min returns $k^* = 1$. Two remarks. With tie rule 1, $k = 2$ always predicts like $k = 1$ (a $1:1$ tie falls back to the nearest point), so $e_2 = e_1$ on every dataset. And distance ties matter on such small data: for $\mathbf{x}_5$ the points $\mathbf{x}_3$, $\mathbf{x}_6$, $\mathbf{x}_8$ are all at squared distance 5; keeping $\mathbf{x}_3$ and $\mathbf{x}_8$ instead would give B at $k = 3$ and a third error.

2.2 Exact search with k-d trees

In low dimension, most of the n distance computations are wasted: points far from the query cannot be neighbours. A *k-d tree* [1, 6] ("k-dimensional tree"; in our notation that k is d, unrelated to the number of neighbours) organises the training points so that whole groups can be skipped. Each node splits its points at the median of one coordinate, cycling through the coordinates with depth; the leaves are small boxes holding a few points (Figure 4). To find the nearest neighbour, descend to the leaf containing the query, take the best point found there as a candidate, and walk back up: the far side of a split need only be searched if the ball around the query whose radius is the current best distance crosses the splitting plane (Algorithm 3).

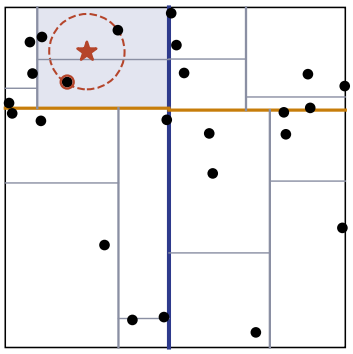


Figure 4. A k-d tree on 24 points, leaf size 2. The ball of radius the best distance in the query's leaf crosses one splitting line: both shaded cells are searched, and the true nearest neighbour (circled) lies in the other one.

The pruning test is safe because the distance from $\mathbf{q}$ to any point on the far side is at least $|q_j - s|$: one coordinate difference never exceeds the full Euclidean distance. For $k > 1$ neighbours the candidate becomes a list of the k best points and ρ the distance to the worst of them.

Algorithm 3 k-d tree: construction and nearest-neighbour search

function BUILD(P, depth) ▷ P: a set of training points; leaf size b
  if $|P| \le b$ **then return** LEAF(P)
  end if
  $j \leftarrow$ depth mod d ▷ or the coordinate of largest spread
  $P_L \leftarrow$ the $\lfloor |P|/2 \rfloor$ points with smallest x_j, $P_R \leftarrow$ the rest ▷ $O(|P|)$ by selection
  $s \leftarrow$ midway between the largest x_j in P_L and the smallest in P_R ▷ rank split: no empty side
  $L \leftarrow$ BUILD(P_L, depth + 1); $R \leftarrow$ BUILD(P_R, depth + 1)
  return NODE(j, s, L, R)
end function
function SEARCH(node, $\mathbf{q}$, best) ▷ query $\mathbf{q}$; best = $(\rho, \mathbf{x})$: best distance so far, initially (∞, none)
  if node is a leaf **then**
    return the better of best and the nearest point of the leaf ▷ brute force on $\le b$ points
  end if
  $(j, s) \leftarrow$ split coordinate and split value stored in node
  (near, far) $\leftarrow$ (left, right) **if** $q_j < s$ **else** (right, left)
  best $\leftarrow$ SEARCH(near, $\mathbf{q}$, best)
  if $|q_j - s| < \rho$ **then** ▷ the ball around $\mathbf{q}$ crosses the plane $x_j = s$
    best $\leftarrow$ SEARCH(far, $\mathbf{q}$, best)
  end if
  return best
end function

Worked example — a k-d tree on the eight points

Build with leaf size $b = 2$. The root splits on x_1: the four smallest values ($\mathbf{x}_4, \mathbf{x}_7$ at 0, $\mathbf{x}_2$ at 1, $\mathbf{x}_1$ at 2) go left, the rest ($\mathbf{x}_3, \mathbf{x}_6$ at 3, $\mathbf{x}_5$ at 4, $\mathbf{x}_8$ at 5) go right, and $s = 2.5$. Both children split on x_2 at $s = 2$ (midway between 1 and 3), giving four leaves: $\{\mathbf{x}_7, \mathbf{x}_2\}$ and $\{\mathbf{x}_1, \mathbf{x}_4\}$ on the left, $\{\mathbf{x}_3, \mathbf{x}_8\}$ and $\{\mathbf{x}_5, \mathbf{x}_6\}$ on the right.
Search for the query $\mathbf{q} = (0.5, 2.5)$. At the root $q_1 = 0.5 < 2.5$, so go left; there $q_2 = 2.5 \ge 2$, so go to the leaf $\{\mathbf{x}_1, \mathbf{x}_4\}$: distances $\sqrt{2.5} = 1.58$ and $\sqrt{0.5} = 0.71$, so $\rho = 0.71$ (candidate $\mathbf{x}_4$). Back at the left node, $|q_2 - 2| = 0.5 < 0.71$: the ball crosses the line $x_2 = 2$, and the leaf $\{\mathbf{x}_7, \mathbf{x}_2\}$ is searched (distances 2.55 and 1.58, no improvement). Back at the root, $|q_1 - 2.5| = 2 \ge 0.71$: the whole right half is pruned. Answer $\mathbf{x}_4$ after 4 distance computations instead of 8. The running query $(2, 2)$ is the unlucky case: it lies on both lines $x_2 = 2$ and only 0.5 from $x_1 = 2.5$, so with $\rho = 1$ (from $\mathbf{x}_1$) no branch can be pruned and all 8 distances are computed.

Running time — k-d tree (Algorithm 3)

Building: the tree has depth $O(\log n)$ and each level does $O(n)$ work for the median selections, so $O(n \log n)$ time; $O(n)$ memory for the nodes on top of the $O(nd)$ data. *Query*: in low dimension and for well-spread data a query visits $O(\log n)$ nodes on average and scans a few leaves of b points, so it costs about $O(\log n + bd)$, or $O(d \log n)$ counting the distances computed on the way — a huge gain over $O(nd)$. As d grows, the ball around the query crosses more and more splitting planes; a common rule of thumb is that the tree helps only when $n \gg 2^d$. For d of a few tens the search visits most leaves and becomes brute force with extra overhead — the curse of dimensionality of Note 3 seen from the algorithmic side.

2.3 Approximate search

Large systems therefore accept *approximate* neighbours. Graph-based indices such as HNSW [12] link each point to a few near neighbours and answer a query by walking greedily through this graph (Algorithm 4); compressed vector codes on GPUs [8] shrink the memory so that billions of vectors fit. Empirically, building such an index costs about $O(n \log n)$ distance evaluations and a query about $O(d \log n)$ instead of $O(nd)$ (there is no worst-case guarantee). Memory is $O(nd)$ for raw vectors, or n times the code size (a few tens of bytes) with compressed codes, plus the links of the graph (Table 1). The price is that a true neighbour is occasionally missed: the index returns *most* of the true neighbours, a fraction called the recall. We only describe these indices; their design and analysis go beyond this course.

Algorithm 4 Greedy search in a neighbour graph (the core step of graph-based indices)

Require: points $\mathbf{x}_1, \dots, \mathbf{x}_n$, each linked to a few near neighbours $G(i)$; an entry point e; query $\mathbf{q}$

$c \leftarrow e$
loop
  $j \leftarrow \arg\min_{i \in G(c)} \|\mathbf{x}_i - \mathbf{q}\|$ ▷ $|G(c)|$ distances, $O(|G(c)|\, d)$
  if $\|\mathbf{x}_j - \mathbf{q}\| \geq \|\mathbf{x}_c - \mathbf{q}\|$ **then return** c ▷ no neighbour is closer: stop
  $c \leftarrow j$
end loop

Running time — greedy graph search (Algorithm 4)

Each step costs $O(Md)$ for a node with M links; the number of steps is typically about $\log n$ in practice (hierarchical layers help), giving roughly $O(Md \log n)$ per query. Memory: the vectors plus $O(nM)$ links. Greedy search can stop at a local minimum that is not the true nearest neighbour, which is where the recall below one comes from; practical indices keep a list of several candidates instead of one.

Modern connection — nearest neighbours inside AI systems

Nearest-neighbour search in embedding space (Note 1, Section 3) is everywhere. *Retrieval-augmented generation* embeds a question, retrieves the nearest passages from a document collection and gives them to a language model as context [11]; the "vector databases" behind such systems are approximate k-NN indices. Updating what the system knows then means adding vectors to the index, not retraining the model. A language model can also be improved without retraining by interpolating its next-token distribution with a k-NN estimate computed from a datastore of contexts [9]: the k retrieved contexts vote for their next tokens with weights proportional to exp(−squared distance), a distance-weighted k-NN vote with tokens as classes (the Gaussian weights of Section 3.2, restricted to $N_k(\mathbf{x})$). And self-supervised vision models are often evaluated by the accuracy of a k-NN classifier on their features [23]: if a simple vote among neighbours works, the representation has organised the data well.

3 Parzen windows: soft neighbourhoods

3.1 Hard windows

Instead of the k nearest points, let every point within a fixed radius h (a radius, not a standard deviation) vote: $w_i(\mathbf{x}) = \mathbf{1}\{d(\mathbf{x}_i, \mathbf{x}) < h\}$ in (1). Equivalently $w_i(\mathbf{x}) = \mathbf{1}\{d(\mathbf{x}_i, \mathbf{x})/h < 1\}$: the distance is measured in units of the window size. The two methods mirror each other:

	k-NN	hard Parzen window
fixed	number of voters k	radius h
varies with $\mathbf{x}$	radius (distance to the k-th neighbour)	number of voters
sparse region	large neighbourhood, still k voters	few or no voters

In sparse regions the window may be empty, and a default prediction is needed, as for histograms (Note 3, Section 1): the overall majority class or mean, or the prediction of the single nearest neighbour.

Algorithm 5 Hard-window Parzen classifier and regressor

Require: training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, radius $h > 0$, a default prediction f_0

function WINDOW($\mathbf{x}$)
  return $N = \{i : \|\mathbf{x}_i - \mathbf{x}\|^2 < h^2\}$ ▷ one pass, $O(nd)$
end function
function CLASSIFYHARD($\mathbf{x}$)
  $N \leftarrow$ WINDOW($\mathbf{x}$); **if** $N = \emptyset$ **then return** f_0 ▷ e.g. majority class or 1-NN
  $v_c \leftarrow |\{i \in N : y_i = c\}|$ for $c = 1, \dots, m$
  return $\arg\max_c v_c$ ▷ ties: class of the nearest member of N (rule 2)
end function
function REGRESSHARD($\mathbf{x}$)
  $N \leftarrow$ WINDOW($\mathbf{x}$); **if** $N = \emptyset$ **then return** f_0 ▷ e.g. overall mean
  return $\frac{1}{|N|} \sum_{i \in N} y_i$
end function

Worked example — hard windows of four sizes

On the data of Figure 1 with query $\mathbf{x} = (2, 2)$, a point votes when $\|\mathbf{x}_i - \mathbf{x}\| < h$, i.e. when $\|\mathbf{x}_i - \mathbf{x}\|^2 < h^2$:

- $h = 0.5$: $h^2 = 0.25$, and no point qualifies (the nearest has squared distance 1). The window is empty and the default rule decides.
- $h = 1.5$: $h^2 = 2.25$, voters $\{1, 2, 3\}$, the same as for $k = 3$: class B, regression $10/3 \approx 3.33$.
- $h = 2.5$: $h^2 = 6.25$, voters $\{1, \dots, 5\}$, the same as for $k = 5$: class A, regression 4.
- $h = 3$: $h^2 = 9$ also admits $\mathbf{x}_7$ (class B, $y_7 = 1$): votes $3 : 3$, a tie, which tie rule 2 of Section 1.3 resolves in favour of the class of the nearest member, $\mathbf{x}_1$: A. Regression $21/6 = 3.5$.

At a different query the same h could collect a different number of points; with k-NN the number is fixed.

3.2 Soft windows and the Nadaraya–Watson estimator

Better still, let *all* training points vote with weights that decrease smoothly with distance, $w_i(\mathbf{x}) = K(\mathbf{x}_i, \mathbf{x})$ for a *kernel* K. The hard window is the special case of a "box" kernel. The most common smooth choice is the Gaussian (radial basis function) kernel

$$K_\sigma(\mathbf{x}_i, \mathbf{x}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}_i\|^2}{2\sigma^2}\right),$$

whose normalising constant $(2\pi\sigma^2)^{-d/2}$ would cancel in the ratio (1) and is therefore omitted. Regression with these weights,

$$f(\mathbf{x}) = \frac{\sum_i K_\sigma(\mathbf{x}_i, \mathbf{x})\, y_i}{\sum_i K_\sigma(\mathbf{x}_i, \mathbf{x})}, \tag{2}$$

is the Nadaraya–Watson estimator [13, 22]; classification uses one-hot targets and an arg max, exactly as for k-NN, so the score of class c is

$$s_c(\mathbf{x}) = \frac{\sum_{i:\, y_i = c} K_\sigma(\mathbf{x}_i, \mathbf{x})}{\sum_i K_\sigma(\mathbf{x}_i, \mathbf{x})}, \qquad \sum_c s_c(\mathbf{x}) = 1.$$

For two classes with labels $y_i \in \{-1, +1\}$ the same read-out is the sign of the weighted mean label,

$$f(\mathbf{x}) = \operatorname{sign} \frac{\sum_i K_\sigma(\mathbf{x}_i, \mathbf{x})\, y_i}{\sum_i K_\sigma(\mathbf{x}_i, \mathbf{x})} = \operatorname{sign}\big(s_{+1}(\mathbf{x}) - s_{-1}(\mathbf{x})\big).$$

Where the formula comes from. The argument of Section 1.1 carries over with weights. The constant c that best fits the data near $\mathbf{x}$, in a *weighted* squared error, solves

$$\frac{d}{dc} \sum_{i=1}^{n} w_i (y_i - c)^2 = -2 \sum_{i=1}^{n} w_i (y_i - c) = 0 \quad \Longrightarrow \quad c = \frac{\sum_i w_i y_i}{\sum_i w_i},$$

which is (2). Nadaraya–Watson is therefore a *locally weighted mean*: at every query a new weighted least-squares problem is solved, with weights centred on the query.

Local linear regression. Replacing the constant c by a linear function centred on the query gives *local linear regression*: at each query $\mathbf{x}$ minimise

$$\sum_{i=1}^{n} w_i(\mathbf{x}) \big(y_i - b - \boldsymbol{\beta}^{\mathsf{T}}(\mathbf{x}_i - \mathbf{x})\big)^2$$

over $b \in \mathbb{R}$ and $\boldsymbol{\beta} \in \mathbb{R}^d$, and predict $f(\mathbf{x}) = b$, the value of the local line at the query. This is a weighted least-squares problem in $d + 1$ unknowns, solved by the normal equations of Note 8, Section 2, with the weights inserted. It removes the first-order bias of Nadaraya–Watson, which is largest at the edges of the data, where all neighbours lie on one side of the query. The price is a new $(d + 1) \times (d + 1)$ linear system for every query: $O(nd^2 + d^3)$ per query instead of $O(nd)$. We mention it for completeness and do not use it further.

Worked example — Gaussian weights and the Nadaraya–Watson prediction

Same data and query $\mathbf{x} = (2, 2)$, Gaussian kernel with $\sigma = 1$, so $w_i = e^{-\delta_i/2}$ with $\delta_i = \|\mathbf{x}_i - \mathbf{x}\|^2$:

i	1	2	3	4	5	6	7	8
class, y_i	A, 5	B, 2	B, 3	A, 4	A, 6	A, 7	B, 1	B, 4
δ_i	1	2	2	5	5	10	8	10
$w_i = e^{-\delta_i/2}$	0.607	0.368	0.368	0.082	0.082	0.007	0.018	0.007

- Total weight $\sum_i w_i = 1.538$.
- Classification: class A collects $0.607 + 0.082 + 0.082 + 0.007 = 0.777$, class B collects $0.368 + 0.368 + 0.018 + 0.007 = 0.761$, so $s_{\mathrm{A}} = 0.777/1.538 = 0.505$ and $s_{\mathrm{B}} = 0.495$. Prediction A, by a hair: the query really is on the boundary.
- Regression: $\sum_i w_i y_i = 5(0.607) + 2(0.368) + 3(0.368) + 4(0.082) + 6(0.082) + 7(0.007) +$

$1(0.018) + 4(0.007) = 5.785$, so $f(\mathbf{x}) = 5.785/1.538 = 3.76$.

With $\sigma = 0.5$ the weights become $e^{-2\delta_i}$: 0.135 for $\mathbf{x}_1$, 0.018 for $\mathbf{x}_2$ and $\mathbf{x}_3$, and less than 10^{-4} for the rest. Now $s_A = 0.787$ and $f(\mathbf{x}) = 4.47$, close to the 1-NN answers (A and 5). Conversely, as $\sigma \to \infty$ every weight tends to 1: $s_A \to 4/8 = 0.5$ and $f(\mathbf{x})$ tends to the mean of all targets, $32/8 = 4$.

The width σ plays the role of k (Figure 5): small widths give fragmented regions that follow individual points, large widths give smooth boundaries that may miss structure. In the limit $\sigma \to 0$ the nearest point dominates and the method becomes 1-NN (Question C5); as $\sigma \to \infty$ it predicts the overall mean or majority. For regression, Figure 6 compares the two families: k-NN regression is a step function, since the neighbour set changes abruptly as the query moves, whereas the Gaussian weights change smoothly and so does the Nadaraya–Watson curve.

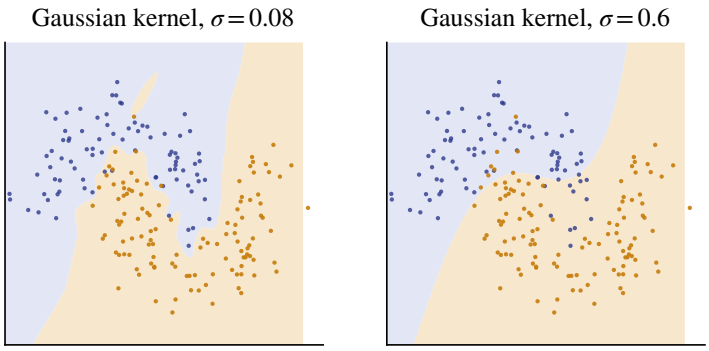


Figure 5. Parzen-window classification of 200 points with a narrow and a wide Gaussian kernel.

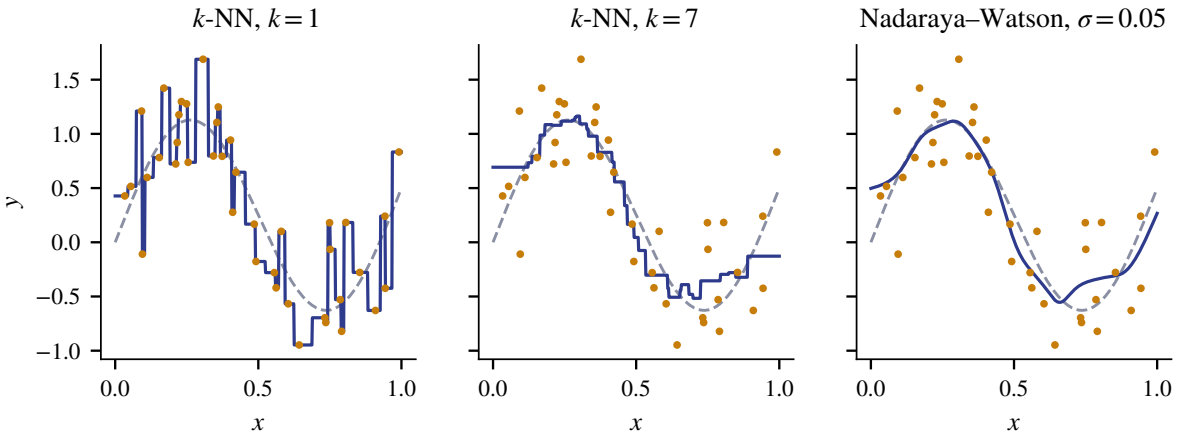


Figure 6. Regression of 40 noisy points (dots) from a smooth function (dashed). Left: 1-NN interpolates the noise. Middle: 7-NN averages it out but is piecewise constant. Right: Nadaraya–Watson with a Gaussian kernel is smooth.

A word on names. "Kernel" here means a *smoothing* kernel, a non-negative bump that sets the weights. Note 10, Section 3, uses the same word for a function $k(\mathbf{x}, \mathbf{x}')$ that computes an inner product in a feature space, and its Section 8 sets the two meanings side by side. The Gaussian kernel is both, which is why the two ideas are often confused; the methods built on them are different.

3.3 Computing the weights safely

For a query far from all data, or a small σ, every weight can underflow to zero and (2) becomes $0/0$. Write $w_i = e^{a_i}$ with $a_i = -\|\mathbf{x}-\mathbf{x}_i\|^2/(2\sigma^2)$ and let $a^* = \max_i a_i$. Multiplying numerator and denominator by e^{-a^*} changes nothing,

$$f(\mathbf{x}) = \frac{\sum_i e^{a_i} y_i}{\sum_i e^{a_i}} = \frac{\sum_i e^{a_i - a^*} y_i}{\sum_i e^{a_i - a^*}},$$

but now every exponent is ≤ 0 and the largest term equals $e^0 = 1$, so the denominator is at least 1: no overflow, no $0/0$. This is the *log-sum-exp trick*; the same identity gives $\log \sum_i e^{a_i} = a^* + \log \sum_i e^{a_i - a^*}$, used for densities in Section 4.

Algorithm 6 Gaussian Parzen classifier and Nadaraya–Watson regressor (log-sum-exp form)

Require: training set $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, width $\sigma > 0$

function WEIGHTS($\mathbf{x}$)
  for $i = 1, \dots, n$ **do**
    $a_i \leftarrow -\|\mathbf{x}-\mathbf{x}_i\|^2/(2\sigma^2)$ ▷ log-weights
  end for
  $a^* \leftarrow \max_i a_i$
  $\omega_i \leftarrow \exp(a_i - a^*)$ for all i ▷ largest ω_i is 1
  return ω
end function
function REGRESS($\mathbf{x}$)
  $\omega \leftarrow$ WEIGHTS($\mathbf{x}$)
  return $\sum_i \omega_i y_i \,/\, \sum_i \omega_i$
end function
function CLASSIFY($\mathbf{x}$)
  $\omega \leftarrow$ WEIGHTS($\mathbf{x}$); $S \leftarrow \sum_i \omega_i$
  $s_c \leftarrow 0$ for $c = 1, \dots, m$
  $s_{y_i} \leftarrow s_{y_i} + \omega_i/S$ for $i = 1, \dots, n$ ▷ scores sum to one
  return $\arg\max_c s_c$ and the scores s
end function

Worked example — underflow and its cure

With $\sigma = 0.02$ on the running example, $a_i = -\delta_i/(2 \cdot 0.02^2) = -1250\,\delta_i$, so $a_1 = -1250$, $a_2 = a_3 = -2500$, and so on. In double precision e^{-1250} is exactly 0 (the smallest positive double is about e^{-744}), so the naive formula returns $0/0$, i.e. `nan`. After subtracting $a^* = -1250$ the weights are $1, e^{-1250}, \dots = 1, 0, \dots, 0$ and $f(\mathbf{x}) = y_1 = 5$: the 1-NN answer, which is indeed the correct limit for a very narrow kernel.

Running time — Parzen windows (Algorithms 5 and 6)

Training: none; memory $O(nd)$ for the stored data. *Prediction per query*: $O(nd)$ for the distances, $O(n)$ for the n comparisons (hard window) or the n exponentials (Gaussian), and $O(n+m)$ to accumulate the class scores (each point adds to its own class). Total $O(nd+m)$, the same order as k-NN but without the selection step. The log-sum-exp trick adds one pass, $O(n)$. A batch of n_q queries is again one $O(n_q nd)$ matrix product followed by element-wise operations, ideal for a GPU. For the hard window in low dimension, a k-d tree answers "all points within radius h" much faster than brute force. A Gaussian kernel can be truncated at a few σ, since $e^{-9/2} \approx 0.011$ at 3σ, after which the same speed-up applies.

Modern connection — attention is kernel smoothing

The attention operation at the heart of the transformer [21] computes, for a query vector $\mathbf{q}$, keys $\mathbf{k}_i$ and values $\mathbf{v}_i$ of dimension d_k,

$$\text{Attention}(\mathbf{q}) = \sum_i \frac{\exp(\mathbf{q}^\mathsf{T}\mathbf{k}_i/\sqrt{d_k})}{\sum_j \exp(\mathbf{q}^\mathsf{T}\mathbf{k}_j/\sqrt{d_k})}\,\mathbf{v}_i,$$

which is (2) with an exponential kernel: each value votes with a weight set by the similarity of its key to the query [20]. The keys play the role of the training inputs $\mathbf{x}_i$, the values that of the targets y_i. Write $\tau = \sqrt{d_k}$ for the temperature. When queries and keys have unit norm, $\exp(\mathbf{q}^\mathsf{T}\mathbf{k}/\tau) = e^{1/\tau}\exp(-\|\mathbf{q}-\mathbf{k}\|^2/(2\tau))$ is proportional to a Gaussian kernel of width $\sigma = \sqrt{\tau}$ in $\|\mathbf{q}-\mathbf{k}\|$; the constant $e^{1/\tau}$ cancels in the ratio (Question C3). The "subtract the maximum" trick of Section 3.3 is the one used in numerically stable softmax implementations. What a transformer adds is that keys, queries and values are *learned* functions of the input, so the network learns which neighbours should vote.

4 Kernel density estimation

For density estimation there are no targets: each training point contributes a small bump of probability, and the estimate is their average [16, 14],

$$\hat{p}(\mathbf{x}) = \frac{1}{n}\sum_{i=1}^{n} K_h(\mathbf{x}-\mathbf{x}_i), \qquad K_h(\mathbf{u}) = \frac{1}{h^d}\,K\!\left(\frac{\mathbf{u}}{h}\right), \tag{3}$$

where $K \geq 0$ is a probability density with $\int K = 1$ (the kernel "shape") and $h > 0$ its width, the *bandwidth*. (For discrete inputs, K is a probability mass function and sums replace the integrals below.) The factor h^{-d} keeps the area equal to one when the bump is stretched by h in each of the d directions. The standard choice is the Gaussian $K(\mathbf{u}) = (2\pi)^{-d/2}e^{-\|\mathbf{u}\|^2/2}$, for which $K_h(\mathbf{x}-\mathbf{x}_i) = \mathcal{N}(\mathbf{x};\mathbf{x}_i,h^2I)$, the isotropic Gaussian of Note 2, Section 4. The bandwidth h plays the role of the width σ of Section 3.2: $K_h(\mathbf{x}-\mathbf{x}_i)$ is $K_\sigma(\mathbf{x}_i,\mathbf{x})$ with $\sigma = h$, now multiplied by its normalising constant $(2\pi h^2)^{-d/2}$. (Do not confuse it with the radius h of the hard window, although that window is the special case of a "box" kernel.) Unlike Nadaraya–Watson, KDE is not a ratio, so here the normalising constant matters.

How KDE relates to the weighted mean. Equation (3) is not an instance of the ratio (1): it is its *denominator* on its own. With weights $w_i(\mathbf{x}) = K\big((\mathbf{x}-\mathbf{x}_i)/h\big)$,

$$\hat{p}(\mathbf{x}) = \frac{1}{nh^d}\sum_{i=1}^{n} w_i(\mathbf{x}) \;:$$

the total weight that the data place near $\mathbf{x}$, normalised by the number of points and the volume of the window, measures how dense the data are there. For the hard window, K is constant on the unit ball and zero outside, and $\hat{p}(\mathbf{x})$ becomes the number of points within radius h divided by n times the volume of that ball: the histogram density of Note 3, Section 1, with the cell centred on the query.

The estimate is a density. Each term is non-negative, so $\hat{p} \geq 0$. For the area, substitute $\mathbf{u} = (\mathbf{x} - \mathbf{x}_i)/h$ in each term, so that $d\mathbf{x} = h^d\, d\mathbf{u}$:

$$\int K_h(\mathbf{x} - \mathbf{x}_i)\, d\mathbf{x} = \int \frac{1}{h^d} K\Big(\frac{\mathbf{x} - \mathbf{x}_i}{h}\Big)\, d\mathbf{x} = \int K(\mathbf{u})\, d\mathbf{u} = 1, \qquad \text{hence} \qquad \int \hat{p}(\mathbf{x})\, d\mathbf{x} = \frac{1}{n}\sum_{i=1}^{n} 1 = 1.$$

Unlike a histogram (Note 3, Section 1), the estimate is smooth and does not depend on where bin edges happen to fall. The density panel of Note 1, Figure 1, was drawn this way. Because $\hat{p}$ is a mixture of n Gaussians with equal weights, it is also easy to *sample* from: pick a training point at random and add Gaussian noise of standard deviation h (Note 2, Section 5).

Worked example — a kernel density estimate by hand

Data $x_1, \ldots, x_4 = 0, 1, 3, 4$ in one dimension, Gaussian kernel, so $\hat{p}(x) = \frac{1}{nh}\sum_i \varphi\big((x - x_i)/h\big)$ with $\varphi(u) = e^{-u^2/2}/\sqrt{2\pi}$. Useful values: $\varphi(0) = 0.3989$, $\varphi(0.5) = 0.3521$, $\varphi(1) = 0.2420$, $\varphi(1.5) = 0.1295$, $\varphi(2) = 0.0540$, $\varphi(4) = 0.0001$.

	$u_i = (x - x_i)/h$	$\sum_i \varphi(u_i)$	nh	$\hat{p}(x)$
$x = 1, h = 0.5$	2, 0, −4, −6	0.4531	2	0.227
$x = 2, h = 0.5$	4, 2, −2, −4	0.1082	2	0.054
$x = 1, h = 2$	0.5, 0, −1, −1.5	1.1225	8	0.140
$x = 2, h = 2$	1, 0.5, −0.5, −1	1.1882	8	0.149

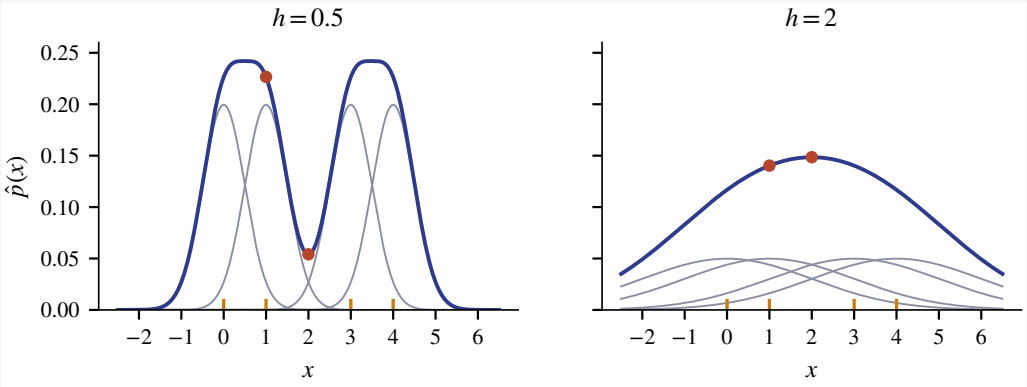


Figure 7. The estimate of this example as a sum of bumps: each data point (tick) contributes $K_h(x - x_i)/n$ (grey), and $\hat{p}$ (indigo) is their sum. Dots: the values at $x = 1$ and $x = 2$ in the table.

With $h = 0.5$ (Figure 7, left) the estimate has two humps, one over the pair 0, 1 (maximum 0.242 near $x \approx 0.52$) and one over the pair 3, 4, and a deep valley at $x = 2$ in the gap (0.227 at $x = 1$ versus 0.054 at $x = 2$, a ratio of about 4). With $h = 2$ the two values are almost equal, and $x = 2$ is in fact the single maximum of the estimate: the gap has been smoothed away. Which picture is right depends on the true density; Section 4.2 shows how to let the data decide.

Running time — kernel density estimation (Algorithm 7)

Training: none (store the data, $O(nd)$ memory). *One density evaluation*: $O(nd)$, like one k-NN query. Evaluating on a grid of G points costs $O(Gnd)$; in one dimension, binning the data and convolving with the kernel by the fast Fourier transform reduces this to roughly $O(n + G \log G)$. In any dimension the G evaluations form one $G \times n$ distance matrix (one matrix product) followed by element-wise exponentials, ideal for a GPU. *One sample*: $O(d)$, independent of n.

Algorithm 7 Gaussian kernel density estimation: log-density and sampling

Require: data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, bandwidth $h > 0$

function LogDensity($\mathbf{x}$)
  for $i = 1, \dots, n$ **do**
    $a_i \leftarrow -\|\mathbf{x} - \mathbf{x}_i\|^2 / (2h^2) - \frac{d}{2}\log(2\pi h^2)$ ▷ $a_i = \log K_h(\mathbf{x} - \mathbf{x}_i)$
  end for
  $a^* \leftarrow \max_i a_i$
  return $a^* + \log \sum_i \exp(a_i - a^*) - \log n$ ▷ log-sum-exp; never $\log 0$
end function
function Sample
  draw i uniformly from $\{1, \dots, n\}$ and $\boldsymbol{\varepsilon} \sim \mathcal{N}(\mathbf{0}, I_d)$
  return $\mathbf{x}_i + h\,\boldsymbol{\varepsilon}$
end function

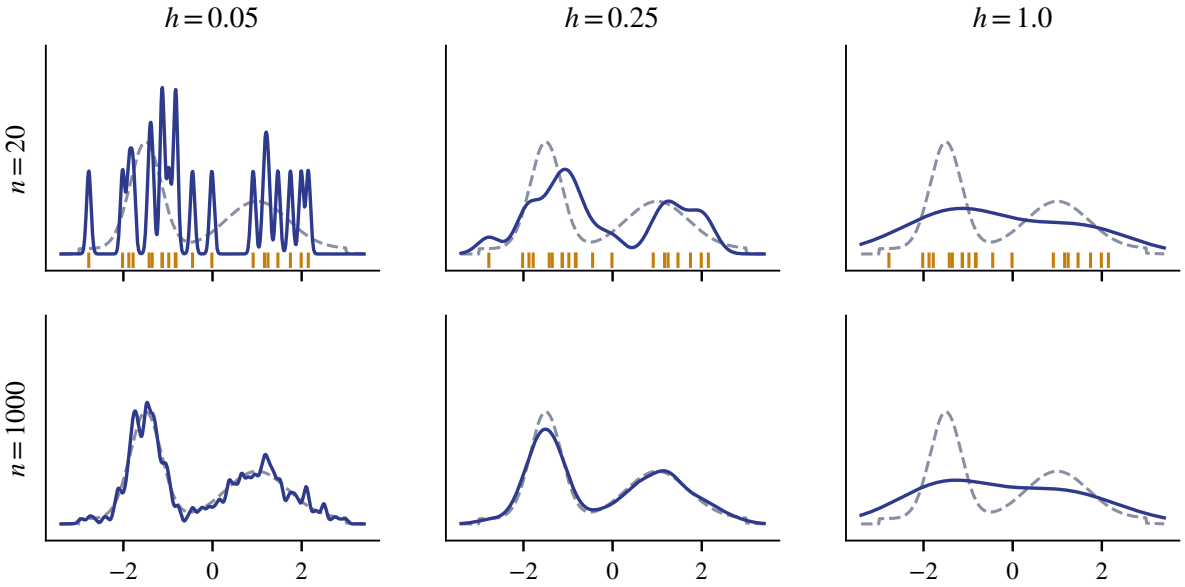


Figure 8. Kernel density estimates (solid) of a two-peaked density with a uniform background (dashed), from $n = 20$ points (top; ticks mark the data) and $n = 1000$ points (bottom), for three bandwidths.

4.1 Bias, variance and the best bandwidth

The bandwidth h trades bias for variance (Figure 8). A narrow kernel puts a spike on each data point; a wide kernel blurs the two peaks into one. More data allow a narrower kernel. The two effects can be computed.

The average estimate is a smoothed density. The terms $Z_i = K_h(\mathbf{x} - \mathbf{X}_i)$ in (3) are independent and identically distributed when the data are, so $\mathbb{E}[\hat{p}(\mathbf{x})] = \mathbb{E}[Z_1]$:

$$\mathbb{E}[\hat{p}(\mathbf{x})] = \int K_h(\mathbf{x} - \mathbf{u})\, p(\mathbf{u})\, d\mathbf{u} = (K_h * p)(\mathbf{x}),$$

the *convolution* of the true density with the kernel. Whatever n is, the estimator on average sees p through a blurred lens of width h; it is unbiased only in the limit $h \to 0$, where K_h approaches a delta function.

Size of the bias. In one dimension, substitute $u = x - ht$ (so $du = -h\,dt$ and $K_h(ht) = K(t)/h$). The minus sign is absorbed by swapping the limits: u running from $-\infty$ to $+\infty$ means t running from $+\infty$ to $-\infty$, and turning the integral back to increasing t costs another minus sign. Then expand p in a Taylor series around x:

$$\mathbb{E}[\hat{p}(x)] = \int K(t)\, p(x - ht)\, dt = \int K(t)\Big[p(x) - ht\, p'(x) + \tfrac{1}{2}h^2t^2 p''(x) - \dots\Big]dt.$$

Use $\int K = 1$, $\int tK(t)\,dt = 0$ for a symmetric kernel, and write $\sigma_K^2 = \int t^2 K(t)\,dt$ (equal to 1 for the Gaussian):

$$\text{bias}(x) = \mathbb{E}[\hat{p}(x)] - p(x) \approx \tfrac{1}{2}\, h^2 \sigma_K^2\, p''(x).$$

At a peak $p'' < 0$ and the estimate is too low; in a valley $p'' > 0$ and it is too high. Peaks are flattened and valleys filled in, by an amount growing like h^2 (Figure 9, left: the peak of height 0.54 is reduced on average to 0.44 with $h = 0.25$ and to 0.29 with $h = 0.6$).

Size of the variance. Since $\hat{p}$ is an average of n independent terms, $\text{Var}[\hat{p}(x)] = \text{Var}(Z_1)/n \le \mathbb{E}[Z_1^2]/n$, and with the same substitution

$$\mathbb{E}[Z_1^2] = \int K_h(x-u)^2 p(u)\, du = \frac{1}{h}\int K(t)^2 p(x - ht)\, dt \approx \frac{p(x)\, R(K)}{h}, \qquad R(K) = \int K(t)^2\, dt.$$

The inequality drops $(\mathbb{E}Z_1)^2/n \approx p(x)^2/n$, which is smaller than $p(x)R(K)/(nh)$ by a factor of order h. Hence $\text{Var}[\hat{p}(x)] \approx p(x)R(K)/(nh)$: few points fall under a narrow kernel, and their number fluctuates.

The best bandwidth. The mean squared error at x is bias squared plus variance (Note 4, Section 6),

$$\text{MSE}(h) \approx \tfrac{1}{4}\sigma_K^4\, p''(x)^2\, h^4 + \frac{p(x)\, R(K)}{nh} = Ah^4 + \frac{B}{nh}.$$

A simple derivative finds the minimum:

$$\frac{d\,\text{MSE}}{dh} = 4Ah^3 - \frac{B}{nh^2} = 0 \quad \Longrightarrow \quad h^* = \left(\frac{B}{4An}\right)^{1/5} \propto n^{-1/5},$$

and inserting h^* gives $\text{MSE}(h^*) \propto n^{-4/5}$, because both terms scale alike: $Ah^{*4} \propto n^{-4/5}$ and $B/(nh^*) \propto n^{-1} \cdot n^{1/5} = n^{-4/5}$. The bandwidth must shrink as n grows, to remove the bias, but slowly, so that $nh \to \infty$ and the variance vanishes too. The rate $n^{-4/5}$ is faster than the $n^{-2/3}$ of histograms in one dimension (Note 3, Section 1). Integrating over x and assuming that p is Gaussian with standard deviation σ_p (the spread of the *data*, not a kernel width) gives, for a Gaussian kernel, $h^* = (4/3)^{1/5}\sigma_p\, n^{-1/5} \approx 1.06\,\sigma_p\, n^{-1/5}$, the *normal-reference rule* of Silverman [17]; in practice σ_p is replaced by the sample standard deviation $\hat{\sigma}_p$.

In d dimensions. The bias is still of order h^2, but a kernel of width h covers a volume h^d, so the variance becomes $B/(nh^d)$. Repeating the derivative for $Ah^4 + B/(nh^d)$ gives $h^* \propto n^{-1/(d+4)}$ and $\text{MSE} \propto n^{-4/(d+4)}$. Ignoring constants, the accuracy that $n = 100$ points give in one dimension ($100^{-4/5} \approx 0.025$) requires in $d = 10$ that $n^{-4/14} = 100^{-4/5}$, i.e. $n = 100^{(4/5)(14/4)} = 100^{2.8} \approx 4 \times 10^5$ points. Kernels soften the curse of dimensionality without lifting it.

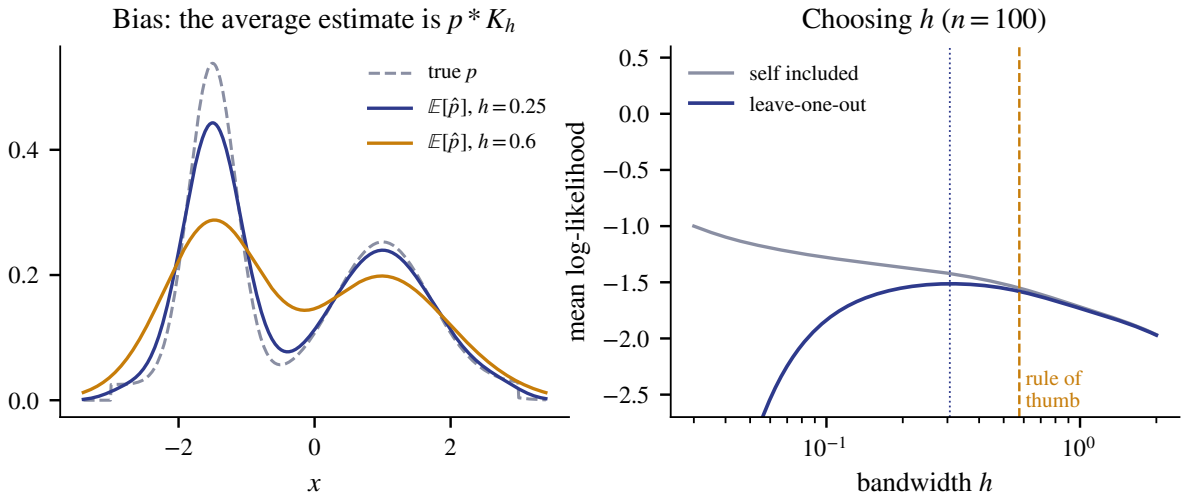


Figure 9. Left: the true density of Figure 8 (dashed) and the average KDE, $K_h * p$, for two bandwidths: peaks are lowered and valleys raised. Right: mean log-likelihood of $n = 100$ points as a function of h, with each point included in its own estimate (grey, increases without limit as $h \to 0$) and left out (indigo, maximal at $h \approx 0.31$, dotted line); the dashed line marks the normal-reference rule, $h \approx 0.58$.

4.2 Choosing the bandwidth from the data

The normal-reference rule is a quick starting point but assumes a single Gaussian bump; for the two-peaked density of Figure 9 it gives $h \approx 0.58$, almost twice the best value, and blurs the peaks. A data-driven choice uses the principle of maximum likelihood (Note 2, Section 2), with care.

Why the plain likelihood fails. Maximising the log-likelihood of the training points themselves, $\sum_i \log \hat{p}(x_i)$, does not work. The estimate at x_i contains the term of x_i itself, so in one dimension

$$\hat{p}(x_i) = \frac{1}{nh} \sum_j \varphi\!\left(\frac{x_i - x_j}{h}\right) \geq \frac{\varphi(0)}{nh}, \qquad \text{hence} \qquad \sum_{i=1}^{n} \log \hat{p}(x_i) \geq n \log \frac{\varphi(0)}{nh} \xrightarrow[h \to 0]{} +\infty.$$

The likelihood is maximised by a spike on every training point, the extreme of overfitting (Figure 9, right, grey curve).

Leave-one-out likelihood. Evaluate each point with the estimate built from the *other* $n - 1$ points,

$$\hat{p}_{-i}(\mathbf{x}_i) = \frac{1}{n-1} \sum_{j \neq i} K_h(\mathbf{x}_i - \mathbf{x}_j), \qquad L_{\mathrm{LOO}}(h) = \sum_{i=1}^{n} \log \hat{p}_{-i}(\mathbf{x}_i),$$

and choose $h^* = \arg\max_h L_{\mathrm{LOO}}(h)$. This is leave-one-out cross-validation (Note 4, Section 4) with the negative log-likelihood as the loss. Now a tiny h is punished: $\hat{p}_{-i}(\mathbf{x}_i)$ is nearly zero unless another point sits right next to $\mathbf{x}_i$, and its logarithm tends to $-\infty$. A huge h is punished too, because it spreads the probability thinly everywhere. The maximum lies in between. (If two data points coincide, as with rounded measurements, $L_{\mathrm{LOO}}(h) \to +\infty$ as $h \to 0$ again: remove duplicates, add jitter below the measurement resolution, or restrict h from below.)

Worked example — leave-one-out bandwidth for four points

Data $0, 1, 3, 4$ as before. With $h = 1$, leaving out $x_1 = 0$, the other points are at distances $1, 3, 4$:

$$\hat{p}_{-1}(0) = \tfrac{1}{3}\big[\varphi(1) + \varphi(3) + \varphi(4)\big] = \tfrac{1}{3}(0.2420 + 0.0044 + 0.0001) = 0.0822,$$

and leaving out $x_2 = 1$ (distances $1, 2, 3$) gives $\frac{1}{3}(0.2420{+}0.0540{+}0.0044) = 0.1001$. By symmetry, x_4 behaves like x_1 and x_3 like x_2, so $L_{\mathrm{LOO}}(1) = 2\log 0.0822 + 2\log 0.1001 = -9.60$. The same computation for other bandwidths (each term now divided by h):

h	$\hat{p}_{-1}(0) = \hat{p}_{-4}(4)$	$\hat{p}_{-2}(1) = \hat{p}_{-3}(3)$	$L_{\mathrm{LOO}}(h)$	$\sum_i \log \hat{p}(x_i)$, self included
0.5	0.0360	0.0361	−13.29	−5.94
1	0.0822	0.1001	−9.60	−7.14
2	0.0893	0.1206	**−9.06**	−8.22
4	0.0775	0.0867	−10.01	−9.79

Leave-one-out picks $h = 2$ among the candidates (a fine grid gives $h^* \approx 1.97$), while the self-included likelihood keeps improving as h decreases, as the bound above predicts. The normal-reference rule, with the sample standard deviation $\hat{\sigma}_p = 1.83$, gives $1.06 \times 1.83 \times 4^{-1/5} = 1.47$, the same order.

Algorithm 8 Bandwidth selection by leave-one-out log-likelihood

Require: data $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$; candidate bandwidths $h_1, \dots, h_H$

$D_{ij} \leftarrow \|\mathbf{x}_i - \mathbf{x}_j\|^2$ for all i, j ▷ once, $O(n^2 d)$
for $\ell = 1, \dots, H$ **do**
  $L_\ell \leftarrow 0$
  for $i = 1, \dots, n$ **do**
    $a_j \leftarrow -D_{ij}/(2h_\ell^2) - \frac{d}{2}\log(2\pi h_\ell^2)$ for all $j \neq i$ ▷ $\log K_h(\mathbf{x}_i - \mathbf{x}_j)$
    $a^* \leftarrow \max_{j \neq i} a_j$
    $L_\ell \leftarrow L_\ell + a^* + \log \sum_{j \neq i} \exp(a_j - a^*) - \log(n-1)$ ▷ $\log \hat{p}_{-i}(\mathbf{x}_i)$
  end for
end for
return h_{ℓ^*} with $\ell^* = \arg\max_\ell L_\ell$

Running time — leave-one-out bandwidth selection (Algorithm 8)

$O(n^2 d)$ for the pairwise distances, then $O(n^2)$ per candidate bandwidth: $O(n^2 d + Hn^2)$ in total. Storing D costs $O(n^2)$ memory — 800 MB in double precision for $n = 10^4$ — so for larger n the rows are recomputed for every candidate ($O(Hn^2 d)$ time, $O(nd)$ memory), or the likelihood is evaluated on a random subset of points. Since $L_{\mathrm{LOO}}(h)$ is usually smooth with a single maximum, a coarse logarithmic grid of 10–20 values of h followed by a refinement suffices. All candidate bandwidths reuse the same matrix D, and each is a batch of element-wise operations on it, which vectorises well on a GPU.

From densities to classifiers. Estimating one density per class with a KDE and combining the estimates with class priors through Bayes' rule gives a classifier (Note 6, Section 2). With n_c points in class c, one common bandwidth $h = \sigma$ and priors equal to the class frequencies n_c/n,

$$\hat{P}(c \mid \mathbf{x}) \propto \frac{n_c}{n} \cdot \frac{1}{n_c} \sum_{i\,:\,y_i = c} K_h(\mathbf{x} - \mathbf{x}_i) = \frac{1}{n} \sum_{i\,:\,y_i = c} K_h(\mathbf{x} - \mathbf{x}_i),$$

and after normalising over c the constants $1/n$ and $(2\pi h^2)^{-d/2}$ cancel: the result is the score $s_c(\mathbf{x})$ of the Gaussian Parzen classifier of Section 3.2.

In physics — unbinned shapes

When a particle-physics analysis needs the distribution of an observable for signal or background, but has only a finite set of simulated events, kernel estimation provides a smooth, unbinned template; adaptive kernels whose width grows in the tails, where events are scarce, are widely used [5]. The adaptive width answers exactly the trade-off above: the relative error of the estimate, $\sqrt{\mathrm{Var}[\hat{p}(x)]}/p(x) \approx \sqrt{R(K)/(nh\,p(x))}$, is largest where p is small, and a wider kernel there brings it down.

5 Distances

Every method in this note is only as good as its notion of "near". A *metric* on a set satisfies, for all $\mathbf{a}, \mathbf{b}, \mathbf{c}$:

1. non-negativity: $d(\mathbf{a}, \mathbf{b}) \geq 0$, with $d(\mathbf{a}, \mathbf{b}) = 0$ if and only if $\mathbf{a} = \mathbf{b}$;
2. symmetry: $d(\mathbf{a}, \mathbf{b}) = d(\mathbf{b}, \mathbf{a})$;
3. triangle inequality: $d(\mathbf{a}, \mathbf{c}) \leq d(\mathbf{a}, \mathbf{b}) + d(\mathbf{b}, \mathbf{c})$.

The triangle inequality is what makes *metric* search structures (ball trees, vantage-point trees) work: it lets an algorithm conclude that a point is far from the query without computing the distance, from the distances to a third point. The k-d tree of Section 2.2 needs less: only the fact that one coordinate difference never exceeds the Euclidean distance.

Minkowski distances. $d_p(\mathbf{a}, \mathbf{b}) = \left(\sum_j |a_j - b_j|^p\right)^{1/p}$ for $p \geq 1$. The cases $p = 1$ (Manhattan), $p = 2$ (Euclidean) and $p \to \infty$ ($\max_j |a_j - b_j|$) are the common ones. Their unit balls (Figure 10) show what each considers "equally far". Always $d_\infty \leq d_2 \leq d_1$: the larger p, the more the distance is dominated by the largest coordinate difference.

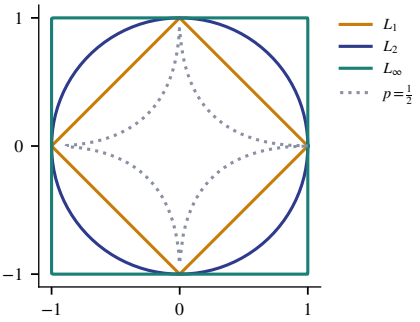


Figure 10. Unit balls $\{\mathbf{a} : d_p(\mathbf{a}, \mathbf{0}) = 1\}$ in $\mathbb{R}^2$.

When $p < 1$. The formula still measures dissimilarity, but it is no longer a metric. Take $\mathbf{a} = (0, 0)$, $\mathbf{b} = (1, 0)$, $\mathbf{c} = (1, 1)$. Then $d_p(\mathbf{a}, \mathbf{b}) = d_p(\mathbf{b}, \mathbf{c}) = 1$ for every p, while

$$d_p(\mathbf{a}, \mathbf{c}) = (1^p + 1^p)^{1/p} = 2^{1/p}.$$

The triangle inequality requires $2^{1/p} \leq 2$, i.e. $1/p \leq 1$, i.e. $p \geq 1$. For $p = 1/2$ the detour through $\mathbf{b}$ has length 2 while the direct route has length $2^2 = 4$. Geometrically, the "unit ball" for $p < 1$ (dotted in Figure 10) is not convex: it is pinched towards the axes.

Similarities for other data. For embeddings and documents, the cosine similarity of Note 1, Section 6, compares directions; as a neighbourhood measure one uses the *cosine distance* $1 - \cos$, which is not a metric (it vanishes for $\mathbf{b} = 2\mathbf{a}$ and can violate the triangle inequality). For binary fingerprints,

such as the substructure bit vectors used to search chemical databases, the Tanimoto (Jaccard) similarity $|\mathbf{a} \wedge \mathbf{b}|/|\mathbf{a} \vee \mathbf{b}|$ counts shared bits relative to bits set in either vector: for $\mathbf{a} = 11010$ and $\mathbf{b} = 10110$ two bits are shared and four are set in either, giving $2/4 = 0.5$. One minus the Tanimoto similarity is a metric on bit vectors.

Worked example — four distances between the same two vectors

Let $\mathbf{a} = (1, 2, 2)$ and $\mathbf{b} = (4, 0, 3)$, so $\mathbf{a} - \mathbf{b} = (-3, 2, -1)$.

- L_1: $3 + 2 + 1 = 6$.
- L_2: $\sqrt{9 + 4 + 1} = \sqrt{14} \approx 3.74$.
- L_∞: $\max(3, 2, 1) = 3$.
- Cosine: $\|\mathbf{a}\| = 3$, $\|\mathbf{b}\| = 5$, $\mathbf{a}^\mathsf{T}\mathbf{b} = 4 + 0 + 6 = 10$, so the cosine similarity $\mathbf{a}^\mathsf{T}\mathbf{b}/(\|\mathbf{a}\|\,\|\mathbf{b}\|)$ is $10/15 = 2/3$ and the cosine distance $1 - 2/3 = 1/3$.

The ordering $3 \le 3.74 \le 6$ illustrates $d_\infty \le d_2 \le d_1$. Replacing $\mathbf{b}$ by $10\mathbf{b}$ leaves the cosine similarity at $2/3$ but changes all three Minkowski distances (the Euclidean one becomes $\sqrt{2309} \approx 48.1$): cosine compares directions only.

When Euclidean distance misleads. Three problems are common, each with a standard remedy.

- *Units and scales.* A feature measured in GeV and another in radians contribute to the distance in proportions set by the units. Standardise each feature, $z_j = (x_j - \hat{\mu}_j)/\hat{\sigma}_j$, with the mean $\hat{\mu}_j$ and standard deviation $\hat{\sigma}_j$ computed on the training set only and then applied unchanged to validation, test and future data (Note 1, Section 5; Algorithm 9).
- *Correlations and irrelevant features.* Correlated features count the same information twice — a duplicated feature doubles its weight in $\|\cdot\|^2$ — and irrelevant ones add noise to every distance. The Mahalanobis distance of Note 2, Section 4, corrects for correlations; feature selection or a learned projection removes irrelevant directions.
- *Invariances.* Two calorimeter images of the same narrow jet, one displaced by a single cell, can be farther apart in pixel space than images of two different jets: the energy deposits no longer overlap. Invariant features (Note 1, Section 3) or a distance that ignores the transformation are needed; Section 5.1 constructs such a distance.

Algorithm 9 Feature standardisation: fit on the training set, apply everywhere (an instance of the Fit/Apply step of Note 1, Section 5)

function Fit($\mathbf{x}_1, \dots, \mathbf{x}_n$) ▷ training inputs only
  for $j = 1, \dots, d$ **do**
    $\hat{\mu}_j \leftarrow \frac{1}{n}\sum_i x_{ij}$; $\hat{\sigma}_j \leftarrow \left(\frac{1}{n}\sum_i (x_{ij} - \hat{\mu}_j)^2\right)^{1/2}$
    if $\hat{\sigma}_j = 0$ **then** $\hat{\sigma}_j \leftarrow 1$ ▷ constant feature: avoid 0/0
  end for
  return $\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\sigma}}$
end function
function Apply($\mathbf{x}$; $\hat{\boldsymbol{\mu}}, \hat{\boldsymbol{\sigma}}$) ▷ training, validation, test and new inputs alike
  return $\mathbf{z}$ with $z_j = (x_j - \hat{\mu}_j)/\hat{\sigma}_j$, $j = 1, \dots, d$
end function

Running time — standardisation (Algorithm 9)

Fit: two passes over the training data, $O(nd)$ time. *Apply*: $O(d)$ per vector (one subtraction and one division per feature). *Memory*: $O(d)$ for $\hat{\boldsymbol{\mu}}$ and $\hat{\boldsymbol{\sigma}}$. Negligible next to any k-NN query, which is why there is no excuse to skip it.

Worked example — the nearest neighbour depends on the units

Two features: an energy E and an angle θ. Query $\mathbf{x} = (50\ \text{GeV}, 0\ \text{rad})$, candidates $\mathbf{x}_A = (52\ \text{GeV}, 1.5\ \text{rad})$ and $\mathbf{x}_B = (47\ \text{GeV}, 0.1\ \text{rad})$.

- Energy in GeV: $d(\mathbf{x}, \mathbf{x}_A) = \sqrt{2^2 + 1.5^2} = 2.5$ and $d(\mathbf{x}, \mathbf{x}_B) = \sqrt{3^2 + 0.1^2} = 3.002$. Nearest: A.
- Energy in TeV: $d(\mathbf{x}, \mathbf{x}_A) = \sqrt{0.002^2 + 1.5^2} \approx 1.500$ and $d(\mathbf{x}, \mathbf{x}_B) = \sqrt{0.003^2 + 0.1^2} \approx 0.100$. Nearest: B.
- Standardised, with training-set standard deviations 10 GeV and 1 rad: the differences become $(0.2, 1.5)$ and $(-0.3, 0.1)$, so $d = 1.513$ and 0.316. Nearest: B.

A change of units, which carries no physical information, changed the answer. Standardisation makes the result independent of units; whether it weights the features *correctly* is a separate question, answered by validation or by learning the metric.

5.1 Invariances and tangent distance

Patterns live on low-dimensional manifolds. Let a pattern $\mathbf{a} \in \mathbb{R}^d$ (an image, a signal) undergo a transformation with r continuous parameters $\boldsymbol{\alpha} \in \mathbb{R}^r$ — a shift across the detector, a rotation, a change of scale or of line thickness — written $F(\mathbf{a}; \boldsymbol{\alpha})$, with $F(\mathbf{a}; \mathbf{0}) = \mathbf{a}$. As $\boldsymbol{\alpha}$ varies, the transformed patterns trace an r-dimensional curved surface, a *manifold*, inside the d-dimensional pixel space. That real data lie near such low-dimensional sets is the reason learning in high dimension is possible at all (Note 3, Section 4). It is also the reason Euclidean distance fails: it measures the straight chord between two points of $\mathbb{R}^d$ and ignores whether one can be moved into the other along its manifold. The distance we want is between the *manifolds* of two patterns, $\min_{\boldsymbol{\alpha},\boldsymbol{\beta}} \|F(\mathbf{a}; \boldsymbol{\alpha}) - F(\mathbf{b}; \boldsymbol{\beta})\|$, but that is a non-linear minimisation for every pair of points.

Tangent distance. Simard, LeCun and Denker [18] replace each manifold by its tangent plane at the pattern. A first-order Taylor expansion in $\boldsymbol{\alpha}$ gives

$$F(\mathbf{a}; \boldsymbol{\alpha}) \approx \mathbf{a} + T\boldsymbol{\alpha}, \qquad T = \left[\mathbf{t}_1, \dots, \mathbf{t}_r\right] \in \mathbb{R}^{d \times r}, \qquad \mathbf{t}_\ell = \left.\frac{\partial F(\mathbf{a}; \boldsymbol{\alpha})}{\partial \alpha_\ell}\right|_{\boldsymbol{\alpha} = \mathbf{0}},$$

where each *tangent vector* $\mathbf{t}_\ell$ is the change of the pattern per unit of the ℓ-th parameter. In practice it is a finite difference, $\mathbf{t}_\ell \approx (F(\mathbf{a}; \varepsilon \mathbf{e}_\ell) - \mathbf{a})/\varepsilon$; for a shift of a sampled signal by s pixels to the right, $a_j(s) = a_{j-s}$, the central difference gives $t_j = (a_{j-1} - a_{j+1})/2$ (zero outside the signal). The *one-sided tangent distance* from $\mathbf{a}$ to $\mathbf{x}$ is

$$d_{\tan}(\mathbf{a}, \mathbf{x}) = \min_{\boldsymbol{\alpha} \in \mathbb{R}^r} \|\mathbf{a} + T\boldsymbol{\alpha} - \mathbf{x}\|\,.$$

The vector $\mathbf{a} + T\boldsymbol{\alpha}$ is linear in $\boldsymbol{\alpha}$, so this is a least-squares problem with r unknowns (Note 8, Section 2): setting the gradient $2T^{\mathsf{T}}(\mathbf{a} + T\boldsymbol{\alpha} - \mathbf{x})$ to zero gives the normal equations

$$(T^{\mathsf{T}}T)\,\boldsymbol{\alpha}^* = T^{\mathsf{T}}(\mathbf{x} - \mathbf{a}), \qquad \text{and for one tangent vector} \qquad \alpha^* = \frac{\mathbf{t}^{\mathsf{T}}(\mathbf{x} - \mathbf{a})}{\mathbf{t}^{\mathsf{T}}\mathbf{t}}.$$

The *two-sided* version also moves $\mathbf{x}$ along its own tangent plane, $\min_{\boldsymbol{\alpha},\boldsymbol{\beta}} \|\mathbf{a} + T_{\mathbf{a}}\boldsymbol{\alpha} - \mathbf{x} - T_{\mathbf{x}}\boldsymbol{\beta}\|$, with $2r$ unknowns; it is symmetric in its two arguments, which the one-sided version is not. Neither is a metric in the sense of the three axioms above (two different patterns can be at distance zero, and the triangle inequality can fail), and both are trustworthy only for *small* transformations, where the plane is close to the manifold; images are often blurred slightly first so that the finite differences are meaningful.

Worked example — tangent distance for a shifted signal

Two training signals of length 5: a "wide" pulse $\mathbf{a} = (0, 1, 2, 1, 0)$ and a "narrow" spike $\mathbf{z} = (0, 0, 0, 2, 0)$. The query $\mathbf{x} = (0, 0, 1, 2, 1)$ is the wide pulse moved one sample to the right.

- Euclidean: $\mathbf{x} - \mathbf{a} = (0, -1, -1, 1, 1)$, so $\|\mathbf{x} - \mathbf{a}\| = \sqrt{4} = 2$; $\mathbf{x} - \mathbf{z} = (0, 0, 1, 0, 1)$, so $\|\mathbf{x} - \mathbf{z}\| = \sqrt{2} = 1.41$. 1-NN says "narrow": wrong.
- Tangent vector of $\mathbf{a}$ for a shift, $t_j = (a_{j-1} - a_{j+1})/2$: $\mathbf{t} = (-0.5, -1, 0, 1, 0.5)$. Then $\mathbf{t}^{\mathsf{T}}\mathbf{t} = 2.5$ and $\mathbf{t}^{\mathsf{T}}(\mathbf{x}-\mathbf{a}) = 0+1+0+1+0.5 = 2.5$, so $\alpha^* = 1$: the best move is one sample to the right, as it should be. The residual is $\mathbf{a} + \mathbf{t} - \mathbf{x} = (-0.5, 0, 1, 0, -0.5)$, so $d_{\tan}(\mathbf{a}, \mathbf{x}) = \sqrt{1.5} = 1.22$.
- Tangent vector of $\mathbf{z}$: $(0, 0, -1, 0, 1)$, orthogonal to $\mathbf{x} - \mathbf{z}$, so $\alpha^* = 0$ and $d_{\tan}(\mathbf{z}, \mathbf{x}) = 1.41$: shifting the spike does not help.

With tangent distance the query is nearer the wide pulse ($1.22 < 1.41$), and 1-NN is right. The residual is not zero although $\mathbf{x}$ *is* an exact shift of $\mathbf{a}$: a one-sample shift is not small for this short signal, and the tangent plane is only a first-order approximation of the manifold.

Algorithm 10 Nearest neighbour with one-sided tangent distance

Require: training points $\mathbf{x}_1, \dots, \mathbf{x}_n \in \mathbb{R}^d$, each with its tangent matrix $T_i \in \mathbb{R}^{d \times r}$

function PREPARE ▷ once, after the tangent vectors are computed
  for $i = 1, \dots, n$ **do**
    $G_i \leftarrow T_i^{\mathsf{T}} T_i$ and its Cholesky factor ▷ $O(dr^2 + r^3)$; add ϵI if singular
  end for
end function
function TANDIST($i, \mathbf{x}$)
  $\mathbf{g} \leftarrow T_i^{\mathsf{T}}(\mathbf{x} - \mathbf{x}_i)$ ▷ $O(dr)$
  solve $G_i\boldsymbol{\alpha} = \mathbf{g}$ with the stored factor ▷ $O(r^2)$
  return $\|\mathbf{x}_i + T_i\boldsymbol{\alpha} - \mathbf{x}\|$ ▷ $O(dr)$
end function
function NEARESTTAN($\mathbf{x}$)
  return $\arg\min_i$ TANDIST($i, \mathbf{x}$) ▷ or the k smallest, as in Algorithm 1
end function

Running time — tangent distance (Algorithm 10)

Notation: r transformations (tangent vectors per point). *Training*: computing the tangent vectors needs r transformed copies of each point, $O(ndr)$ if a transformation costs $O(d)$; PREPARE adds $O(n(dr^2 + r^3))$. *Memory*: $O(ndr)$ for the tangents (with the data, $r + 1$ times the raw data) plus $O(nr^2)$ for the factors. *Prediction per query*: $O(n(dr + r^2))$, about r times a Euclidean query. The two-sided version depends on the query's own tangents, so its $2r \times 2r$ system must be formed and solved for every pair: $O(n(dr^2 + r^3))$ per query. A common speed-up is to rank all points by the cheap Euclidean distance first and to compute the tangent distance only for the few hundred nearest.

Modern connection — learning the metric

Today the distance is usually learned. Contrastive methods train a network so that two transformed views of the same image map to nearby embeddings and views of different images map far apart [3]; image–text models do the same for an image and its caption [15]. The invariances that tangent distance builds in by hand are learned from the transformations used in training, and the plain cosine or Euclidean distance in the learned space then works well for k-NN, retrieval and clustering.

In physics — a metric for collider events

Komiske, Metodiev and Thaler defined the distance between two collider events as the minimal "work" needed to rearrange one event's energy flow into the other's — an optimal-transport, or earth mover's, distance — which respects the physical requirement that soft or collinear emissions should change it only slightly [10]. With this metric, even a simple k-NN classifier separates jet types well, and geometric quantities of the space of events become measurable. Choosing the distance is where the physics enters.

6 Neighbourhood methods at a glance

Table 1 collects the costs derived above. All methods share the same profile — no training, $O(nd)$ memory, prediction linear in n — and differ in how the neighbourhood is defined and in what extra structure (a tree, an index) is bought to speed up the search.

Table 1. Hyperparameters and costs of the methods of this note; n examples, d features, m classes, $k_{\max}$ candidate values of k, H candidate bandwidths, r tangent vectors per training point.

Method	Hyperparameters	Training	Prediction per query	Memory
k-NN, brute force (Alg. 1)	k, distance	none	$O(nd + k\log k + m)$	$O(nd)$
k-NN with a k-d tree (Alg. 3)	k, distance, leaf size	$O(n\log n)$	about $O(d\log n)$ for small d; up to $O(nd)$	$O(nd) + O(n)$ tree
k-NN, approximate index (Alg. 4)	k, distance, index settings	about $O(n\log n)$ distances	about $O(d\log n)$, approximate	$O(nd)$ or codes, + links
k-NN, tangent distance (Alg. 10)	k, transformations	$O(n(dr^2 + r^3))$	$O(n(dr + r^2))$	$O(ndr)$
Hard Parzen window (Alg. 5)	radius h, distance	none	$O(nd + m)$	$O(nd)$
Gaussian Parzen, Nadaraya–Watson (Alg. 6)	width σ	none	$O(nd + m)$	$O(nd)$
KDE (Alg. 7)	bandwidth h	none	$O(nd)$; sampling $O(d)$	$O(nd)$
Choose k by leave-one-out (Alg. 2)	$k_{\max}$	$O(n^2d + nk_{\max}\log k_{\max})$	—	$O(nd)$
Choose h by leave-one-out (Alg. 8)	$h_1, \dots, h_H$	$O(n^2d + Hn^2)$	—	$O(n^2)$ for D, or $O(nd)$

Summary k-NN and Parzen windows predict by a weighted vote of training examples, with weights given by neighbourhood membership or by a kernel; the same formula — the weighted mean that minimises a weighted squared error — serves regression and classification, and its denominator, the total weight near the query, normalised by n and the window volume, is the kernel density estimate. They need no training, are consistent as $n \to \infty$ provided the neighbourhood shrinks slowly enough ($k \to \infty$ with $k/n \to 0$, or $h \to 0$ with $nh^d \to \infty$), whereas 1-NN only reaches at most twice the Bayes error, and they have one main hyperparameter — k, the radius or the bandwidth — that trades bias for variance; for KDE the bias is $K_h * p - p \approx \frac{1}{2}h^2\sigma_K^2 p''$ and the variance $\approx pR(K)/(nh)$, giving $h^* \propto n^{-1/5}$, and leave-one-out (never the plain training likelihood) selects k or h from data. Prediction costs $O(nd)$ per query unless a tree (low d) or an approximate index (high d) is used, and the log-sum-exp trick keeps kernel weights finite. Their weak point is the distance: it must reflect the problem's units, correlations and invariances (standardisation, Mahalanobis distance, tangent distance), which is why modern systems learn the representation and then use simple neighbourhood methods on top of it.

Questions and exercises

Questions are sorted by kind. Parts A and B are of the kind used in the midterm examination: they test understanding, interpretation and short calculations (at most simple derivatives), not long proofs. Parts C and D go further and are recommended for practice and for the project.

A. Conceptual questions

A1. k-NN is called a "lazy" or non-parametric method. Explain what is stored after training, what training costs, and why predicting one query is expensive compared with a model that stores only a fixed vector $\mathbf{w} \in \mathbb{R}^d$ and predicts $\mathbf{w}^\mathsf{T}\mathbf{x}$ at cost $O(d)$.

A2. What is the training error of 1-NN on a dataset in which no input appears twice? Why is this number useless for choosing k, and what should be used instead?

A3. Why can leave-one-out for k-NN produce the validation error for every $k = 1, \dots, k_{\max}$ in a single pass over the data, whereas for most models leave-one-out requires n separate trainings?

A4. Explain how k controls bias and variance in k-NN. What does k-NN predict when $k = n$, in classification and in regression?

A5. True or false, with a justification: "For two classes, an odd k removes every ambiguity from the k-NN rule." Does the answer change for three classes?

A6. Compare k-NN and the hard Parzen window: which one has a neighbourhood of fixed size and which one a fixed number of voters? Describe what each does for a query far from all training data and for a query in a very dense region.

A7. Describe the Gaussian Parzen classifier in the two limits $\sigma \to 0$ and $\sigma \to \infty$. Which classifier, or which constant prediction, does it become in each case?

A8. For a very narrow Gaussian kernel, a direct implementation of the Nadaraya–Watson formula returns `nan`. Explain why, and why subtracting the largest exponent from all exponents fixes the problem without changing the prediction.

A9. The normalising constant of the Gaussian kernel can be dropped in the Nadaraya–Watson

estimator but not in kernel density estimation. Explain why.

A10. Why is k-NN regression a step function of the query, while Nadaraya–Watson with a Gaussian kernel gives a smooth curve (Figure 6)?

A11. A student chooses the KDE bandwidth by maximising the log-likelihood of the training points themselves. Which bandwidth will she obtain, and why? How does leave-one-out repair this?

A12. Explain the bias of a kernel density estimate in words: what happens to a sharp peak and to a valley of the true density when h is large? Why does the variance grow when h is small? The best bandwidth scales as $n^{-1/5}$: explain why it must decrease as n grows, and why it must not decrease too fast.

A13. True or false, with a justification: (a) a kernel density estimate can take values larger than 1; (b) in exact arithmetic, a Gaussian KDE is never exactly zero; (c) a KDE integrates to one only if the bandwidth is chosen well.

A14. One feature is a length in metres (values between 0 and 1), another a mass in grams (values between 0 and 1000). What goes wrong with Euclidean k-NN on the raw features? Propose a remedy and explain why its constants must be computed on the training set only.

A15. Is the cosine distance $1 - \cos(\mathbf{a}, \mathbf{b})$ a metric on $\mathbb{R}^d$? Which of the three metric axioms fail, and for which kind of pairs of vectors? Why can it nevertheless be a good choice for k-NN on text embeddings?

A16. What happens to Euclidean distances, and hence to k-NN, if (a) one feature is accidentally included twice, (b) twenty features of pure noise are appended?

A17. Why does a k-d tree speed up nearest-neighbour search in two dimensions but hardly help in $d = 100$? Relate your answer to the curse of dimensionality of Note 3.

A18. An approximate index returns, on average, 95% of the true k nearest neighbours. What is gained and what is lost compared with brute-force search? Why can greedy search in a neighbour graph miss the true nearest neighbour?

A19. What does the Cover–Hart result say about the 1-NN classifier as $n \to \infty$, and what does it *not* say about a finite dataset in high dimension?

A20. In retrieval-augmented generation, which step is a nearest-neighbour search, and what are the "training points"? Why is it attractive to update the document collection rather than retrain the language model?

A21. In what sense is the attention operation of a transformer a Nadaraya–Watson estimator? What plays the role of the training inputs, of the targets and of the kernel width?

A22. Images of one object under small shifts or rotations lie near a low-dimensional manifold in pixel space. Explain why the Euclidean distance between two such images can be large, what a tangent vector describes, and what tangent distance minimises. Why does it work only for small transformations?

A23. The kernel density estimate is not a weighted mean of targets. How is it nevertheless related to the Nadaraya–Watson formula? What does the denominator $\sum_i K_h(\mathbf{x} - \mathbf{x}_i)$ measure?

B. Short calculations

B1. Explain why the 1-NN decision boundary between two training points of different classes lies on their perpendicular bisector. Sketch the Voronoi cells and the decision boundary for the four points $(0,0)$, $(2,0)$, $(0,2)$, $(2,2)$ labelled $+,-,-,+$.

B2. One-dimensional training data: inputs $0,1,2,4,5,7$ with classes $+,+,-,+,-,-$ and regression targets $1.0, 1.5, 2.5, 3.0, 2.0, 0.5$. For the query $x = 3.2$, list the distances, give the k-NN class for $k = 1, 3, 5$ and the k-NN regression value for $k = 3$.

B3. Training inputs $0,1,3$ with targets $2,4,1$. At the query $x = 1$, compute (a) the hard-window prediction with radius $h = 1.5$, (b) the Nadaraya–Watson prediction with the Gaussian kernel of width $\sigma = 1$, $e^{-(x-x_i)^2/(2\sigma^2)}$, (c) the limit of this prediction as $\sigma \to 0$.

B4. Data $-1, 0, 0, 2$ (the value 0 appears twice). Using $\varphi(0) = 0.399$, $\varphi(1) = 0.242$, $\varphi(2) = 0.054$, compute the Gaussian KDE with $h = 1$ at $x = 0$ and at $x = 1$. Repeat with the box kernel $K(u) = \frac{1}{2}\mathbf{1}\{|u| \le 1\}$ and $h = 1$ (note the "$\le$": a point at distance exactly 1 counts; this box has width 2, whereas the one of Question C1 has width 1; both integrate to one).

B5. For $\mathbf{a} = (2,-1,0,3)$ and $\mathbf{b} = (0,1,1,1)$ compute d_1, d_2, d_∞ and the cosine similarity. For the bit vectors 1101100 and 0101110 compute the Tanimoto similarity.

B6. The mean squared error of a KDE behaves like $\mathrm{MSE}(h) = Ah^4 + B/(nh)$ with $A, B > 0$. (a) Find the minimising h^* by differentiation. (b) Show by substituting h^* that the minimal MSE is proportional to $n^{-4/5}$. By what factor does h^* change when n is multiplied by 32, and by what factor does the minimal MSE change? (c) Repeat (a) for $Ah^4 + B/(nh^d)$.

B7. A retrieval system stores 10^7 embeddings in $\mathbb{R}^{768}$ as 32-bit floats. How much memory do they take, and how many multiply–add operations does one brute-force query need? At 10^{12} multiply–adds per second, how many queries per second is that?

B8. Query (300 K, 1.0 bar); candidates P = (305 K, 1.0 bar) and Q = (300 K, 3.0 bar). Which candidate is nearer in raw units? Which after standardising with training standard deviations 10 K and 0.5 bar?

B9. A training signal $\mathbf{a} = (0,2,2,0)$ and a query $\mathbf{x} = (0,0,2,2)$, the same signal shifted by one sample to the right. (a) Compute $\|\mathbf{x} - \mathbf{a}\|$. (b) Compute the shift tangent vector $t_j = (a_{j-1} - a_{j+1})/2$, with $a_0 = a_5 = 0$. (c) Find $\alpha^* = \mathbf{t}^\mathsf{T}(\mathbf{x} - \mathbf{a})/\mathbf{t}^\mathsf{T}\mathbf{t}$ and the one-sided tangent distance $\|\mathbf{a} + \alpha^*\mathbf{t} - \mathbf{x}\|$. Interpret α^*, and explain why the result is not zero.

C. Derivations

C1. Let K be the box kernel, $K(u) = 1$ for $|u| \le \frac{1}{2}$ and 0 otherwise, in one dimension. Show that the KDE integrates to one and that $\mathbb{E}[\hat{p}(x)] = \frac{1}{h}\int_{x-h/2}^{x+h/2} p(u)\,du$. Compute σ_K^2 and $R(K)$, and deduce the approximate bias $h^2 p''(x)/24$ and variance $p(x)/(nh)$. Compare with the histogram of Note 3: what is gained by centring the window on x?

C2. Show that d_p violates the triangle inequality for every $0 < p < 1$ (you may use the points of Section 5), and that the set $\{\mathbf{a} \in \mathbb{R}^2 : d_p(\mathbf{a}, \mathbf{0}) \le 1\}$ is not convex for $p < 1$ by exhibiting two of its points whose midpoint lies outside it.

C3. Let $\|\mathbf{q}\| = \|\mathbf{k}_i\| = 1$. Show that $\exp(\mathbf{q}^\mathsf{T}\mathbf{k}_i/\tau)$ is proportional to a Gaussian kernel in $\|\mathbf{q} - \mathbf{k}_i\|$ and find its width. What happens to attention as $\tau \to 0$?

C4. Show that, for weights $w_i \geq 0$, (a) the weighted mean minimises $\sum_i w_i (y_i - c)^2$, (b) the weighted majority class minimises $\sum_i w_i \mathbf{1}\{y_i \neq c\}$ over classes c, and (c) a weighted median minimises $\sum_i w_i |y_i - c|$. (A weighted median is a value c such that the targets below c and the targets above c each carry at most half of the total weight $\sum_i w_i$. The absolute value has no derivative at y_i, so instead of setting a derivative to zero, consider how $\sum_i w_i |y_i - c|$ changes as c moves between two consecutive targets.) Which read-out is least sensitive to one wrong target?

C5. Assume the nearest training point to $\mathbf{x}$ is unique, with index i^*. Using the log-sum-exp form of Section 3.3, show that the normalised Gaussian weights $\omega_i / \sum_j \omega_j$ tend to $\mathbf{1}\{i = i^*\}$ as $\sigma \to 0$, so that the Gaussian Parzen classifier and regressor become 1-NN. Show also that all normalised weights tend to $1/n$ as $\sigma \to \infty$.

D. Programming

D1. Implement Algorithm 1 with numpy and generate data of your own with two interleaved classes. Choose k by 5-fold cross-validation and by Algorithm 2, and plot both validation curves against k. How does the selected k change when you double n?

D2. Implement Algorithm 6. Reproduce the worked examples of Section 3.2 and Section 3.3 (including $\sigma = 0.02$, and the `nan` produced without the log-sum-exp trick). On noisy samples of $\sin(2\pi x)$, choose k for k-NN regression and σ for Nadaraya–Watson by validation and compare their test errors.

D3. Implement Algorithms 7 and 8. Draw 100 points from a mixture of two Gaussians, reproduce the right panel of Figure 9, and compare the leave-one-out bandwidth with the normal-reference rule for well-separated and for overlapping components.

D4. Implement Algorithm 3 for the single nearest neighbour, verify it against brute force on random data, and count the leaves visited per query for $n = 10^4$ uniform points in $d = 2, 5, 10, 20$. At which d does the tree stop paying off?

Further reading. Hastie, Tibshirani and Friedman [7], sections 2.3, 6.1–6.6 and 13.3; Bishop [2], section 2.5; Silverman [17] for kernel density estimation and bandwidth selection.

References

[1] J. L. Bentley, Multidimensional binary search trees used for associative searching, *Commun. ACM* 18, 509–517 (1975).

[2] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer (2006).

[3] T. Chen, S. Kornblith, M. Norouzi and G. Hinton, A simple framework for contrastive learning of visual representations, *ICML* (2020).

[4] T. M. Cover and P. E. Hart, Nearest neighbor pattern classification, *IEEE Trans. Inf. Theory* 13, 21–27 (1967).

[5] K. Cranmer, Kernel estimation in high-energy physics, *Comput. Phys. Commun.* 136, 198–207 (2001).

[6] J. H. Friedman, J. L. Bentley and R. A. Finkel, An algorithm for finding best matches in logarithmic expected time, *ACM Trans. Math. Softw.* 3, 209–226 (1977).

[7] T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer (2009).

[8] J. Johnson, M. Douze and H. Jégou, Billion-scale similarity search with GPUs, *IEEE Trans. Big Data* 7, 535–547 (2021).

[9] U. Khandelwal *et al.*, Generalization through memorization: nearest neighbor language models, *ICLR* (2020).

[10] P. T. Komiske, E. M. Metodiev and J. Thaler, Metric space of collider events, *Phys. Rev. Lett.* 123, 041801 (2019).

[11] P. Lewis *et al.*, Retrieval-augmented generation for knowledge-intensive NLP tasks, *NeurIPS* (2020).

[12] Y. A. Malkov and D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 824–836 (2020).

[13] E. A. Nadaraya, On estimating regression, *Theory Probab. Appl.* 9, 141–142 (1964).

[14] E. Parzen, On estimation of a probability density function and mode, *Ann. Math. Statist.* 33, 1065–1076 (1962).

[15] A. Radford *et al.*, Learning transferable visual models from natural language supervision, *ICML* (2021).

[16] M. Rosenblatt, Remarks on some nonparametric estimates of a density function, *Ann. Math. Statist.* 27, 832–837 (1956).

[17] B. W. Silverman, *Density Estimation for Statistics and Data Analysis*, Chapman & Hall (1986).

[18] P. Simard, Y. LeCun and J. S. Denker, Efficient pattern recognition using a new transformation distance, *NeurIPS* (1993).

[19] C. J. Stone, Consistent nonparametric regression, *Ann. Statist.* 5, 595–620 (1977).

[20] Y.-H. H. Tsai *et al.*, Transformer dissection: an unified understanding for transformer's attention via the lens of kernel, *Proc. EMNLP-IJCNLP*, 4344–4353 (2019).

[21] A. Vaswani *et al.*, Attention is all you need, *NeurIPS* (2017).

[22] G. S. Watson, Smooth regression analysis, *Sankhyā A* 26, 359–372 (1964).

[23] Z. Wu, Y. Xiong, S. X. Yu and D. Lin, Unsupervised feature learning via non-parametric instance discrimination, *CVPR* (2018).

The selection and order of topics in these notes follow the course IFT 6390 *Fondements de l'apprentissage machine* at Université de Montréal (taught by Ioannis Mitliagkas, slides by Pascal Vincent). The text, examples, figures and exercises of this note are original. Figures are computed from synthetic data by `figures.py`, published with the sources of these notes.